



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
ICS2121 – MÉTODOS DE OPTIMIZACIÓN

Apuntes de Métodos de Optimización

*Por Francisco García Aubert y Jorge Vera Andreo
Material en desarrollo. No compartir fuera del curso.
(versión 7, Agosto 2025)*



Índice general

1. Introducción: problemas y algoritmos	3
1.1. El problema general	3
1.2. Soluciones aproximadas	3
1.3. Algoritmos y oráculos	4
2. Métodos del Gradiente, Newton y extensiones	7
2.1. La idea del Algoritmo de Descenso	7
2.2. Método del Gradiente	8
2.3. Método de Newton	12
2.4. Midiendo la eficiencia de un algoritmo	14
2.4.1. Convergencia del Método del Gradiente	17
2.4.2. Cambios de variable para mejorar el condicionamiento	19
2.4.3. Velocidad de convergencia para el Método de Newton	20
2.4.4. Costo por iteración del Método del Gradiente y Newton	21
2.5. Linesearch: Selección del paso	22
2.5.1. Métodos de búsqueda	22
2.5.2. Condición de Wolfe-Armijo	24
2.6. Métodos Quasi-Newton	25
2.6.1. Métodos Quasi-Newton: BFGS	25
2.6.2. Eficiencia de BFGS	26
2.7. Ejemplos	28
2.7.1. Aplicación en IA: Optimización de Redes Neuronales	28
2.7.2. Aplicación: Optimización en doblamiento de proteínas	31
2.7.3. Centro Analítico de un Poliedro	34
2.7.4. Localización espacial	36
2.7.5. Estimaciones Estadísticas	38
2.7.6. Función Banana	40
2.7.7. Un algoritmo aproximado con estructura “tipo Newton”	43

Capítulo 1

Introducción: problemas y algoritmos

En este capítulo pretendemos dar una introducción y motivación muy general a la necesidad de desarrollar métodos computacionales para abordar problemas de optimización. Las personas que estudian cálculo y temas básicos de optimización pueden quedarse con la idea de que para resolver un problema basta “derivar e igualar a cero”. Como veremos a lo largo de este curso, eso es una falacia: prácticamente ningún problema real se puede abordar de esa forma. La complejidad de los problemas requerirá el desarrollo de métodos iterativos como única forma de llegar, no a una solución del problema, si que a una aproximación suficientemente buena.

1.1. El problema general

El problema de optimización general se denota típicamente como:

$$P) \quad \min_{x \in S} f(x)$$

Donde x es un vector que está en $\mathbb{R}^n$, $f : \mathbb{R}^n \rightarrow \mathbb{R}$ y el conjunto $S \subset \mathbb{R}^n$ corresponde al dominio. Este conjunto S se puede explicitar:

$$\begin{aligned} P) \quad & \min f(x) \\ \text{s.a.} \quad & g_i(x) \leq b_i, \quad i = 1, \dots, m \end{aligned}$$

Donde $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $g_i : \mathbb{R}^n \rightarrow \mathbb{R}$ y $S \subset \mathbb{R}^n$. Si bien, en la mayoría de los casos haremos explícitas las “restricciones”, a veces el conjunto S no es describible a través de fórmulas, pero sí matemáticamente, tal como las restricciones de naturaleza de las variables, ej: $x_i \in \{0, 1\}$ y otros más complejos.

1.2. Soluciones aproximadas

Supongamos, de momento, que $S = \mathbb{R}^n$, es decir, no hay restricciones, y que la función f puede ser derivada sin complicaciones. En ese caso, la solución x^* que se busca corresponde al mínimo global, valor que se obtiene donde $\nabla f(x^*) = 0$. El problema de esto es que en muchos problemas existen múltiples mínimos locales donde el gradiente de la función también se anula, también el hecho que pueden existir ciertas restricciones (y la condición nos e cumple) y que muchas funciones son de miles o millones de variables. De acá la insuficiencia de simplemente derivar una función e igualar a cero. Más aún, muchas veces en muchas funciones, al derivar e igualar a cero no se llega a ninguna parte útil, tal como en el siguiente caso:

$$f(x) = x^2 + e^{-x}$$

Derivando e igualando a cero:

$$f'(x) = 2x - e^{-x} = 0 \longrightarrow x = \frac{1}{2}e^{-x}$$

En esta última ecuación no se puede despejar x por medio de fórmulas simple algebraicas (los matemáticos llaman a esto una ecuación trascendente) y se debe optar por otro método para encontrar el mínimo. Se puede resolver llegando a una solución aproximada, lo que se puede hacer con algún algoritmo iterativo, como lo es el de punto fijo, donde en cada iteración se aproxima más a la solución óptima. El algoritmo consiste en partir de una solución arbitraria para x , llamémosla x^0 . Con esta se obtiene x^1 resolviendo $x^1 = 1/2e^{-x^0}$, luego se resuelve $x^2 = 1/2e^{-x^1}$ y así recursivamente:

$$x^{k+1} = \frac{1}{2}e^{-x^k}$$

A medida que se itera se va alcanzando una situación de convergencia, por lo que se repite hasta alcanzar un valor que este cercano al óptimo x^* . La moraleja es que derivar e igualar a cero generalmente es totalmente inútil, sino que se debe optar por un enfoque iterativo aproximado.

Formalicemos los objetivos que buscamos. Se dice que $\bar{x} \in S$ es una solución **ϵ -óptima** de P) si para un $\epsilon \geq 0$ su valor objetivo se diferencia en menos de ϵ del valor óptimo:

$$\|f(\bar{x}) - f(x^*)\| \leq \epsilon$$

Por otro lado, se dice que una solución $\bar{x}$ es **ϵ -aproximada** si la distancia o norma de esta solución con la solución óptima es menor a ϵ :

$$\|\bar{x} - x^*\| \leq \epsilon$$

Con una solución ϵ -óptima estamos focalizándonos en que el objetivo sea lo más cercano posible a su valor mínimo. Por otro lado, la solución ϵ -aproximada se focaliza en como “producir” el vector x de modo que sea lo más parecido posible a la combinación de valores de variables que dan el óptimo. Ambos conceptos no son equivalentes pero están relacionados:

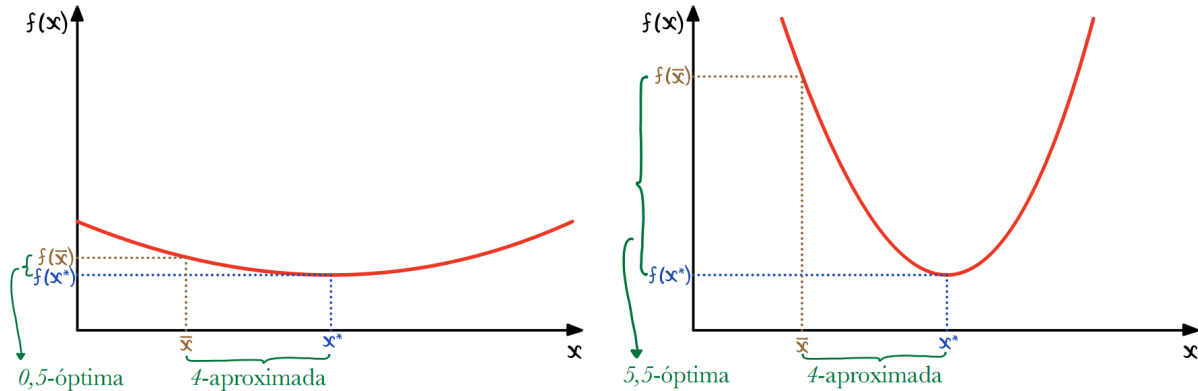


Figura 1.1: Solución épsilon óptima y épsilon aproximada

En el ejemplo de la imagen se puede ver que si el gráfico de la función en el óptimo es muy plano, no se requiere una solución ϵ -aproximada muy precisa para obtener una buena solución ϵ -óptima.

1.3. Algoritmos y oráculos

La idea general consiste principalmente iterar por medio de algoritmos de optimización, para así ir generando una sucesión de puntos $x^0, x^1, x^2, \dots$. Para generar los nuevos puntos se utilizan las iteraciones anteriores y una subrutina o “**oráculo**” que entrega información de la función f :

Oráculo/información	Valores	Derivadas	Segundas derivadas
Orden cero	✓		
Primer orden	✓	✓	
Segundo orden	✓	✓	✓

Al momento de utilizar estos oráculos, se debe tener en cuenta que acceder a más información es más costoso en tiempo de cómputo, por lo que no necesariamente un oráculo de segundo orden es más conveniente que uno de primer orden, ni este más conveniente que uno de orden cero. Es acá donde surge el problema de la eficiencia del algoritmo de optimización: Dado un error aceptable $\epsilon > 0$ ¿cuánto tiempo necesitamos en la iteración de un algoritmo para obtener este error?

En el caso del Oráculo de orden cero, que solo entrega valores de la función f , se pueden evaluar puntos de f para ver en dónde se encuentran valores objetivos buenos. El problema es que no necesariamente el óptimo se encuentra aledaño a los mejores puntos encontrados, tal como en el problema $\min_{x \in S} f(x)$ de la Figura 1.2.

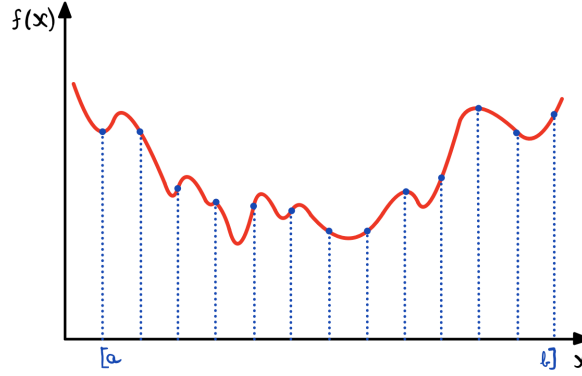


Figura 1.2: Puntos encontrados por un oráculo de orden cero

El problema acá es que la función puede oscilar mucho y el óptimo no necesariamente se puede captar. Este problema se puede solucionar si se sabe que la función f satisface una **condición de Lipschitz** con constante L :

$$|f(x) - f(y)| \leq L|x - y|, \quad \forall x, y$$

Esta condición dice que para dos puntos x e y , la diferencia de sus valores objetivos se encuentra acotada en forma proporcional a la distancia entre x e y . Más aún, todo valor objetivo de una solución que se encuentre entre x e y está acotado. Esto se debe a que la derivada (y por tanto la pendiente) de la función entre cualquier par de puntos está acotada por L , para ver esto, se divide la expresión anterior por $|x - y|$ y se toma el límite cuando x está muy cercano a y :

$$\lim_{y \rightarrow x} \frac{|f(x) - f(y)|}{|x - y|} \leq L, \quad \forall x$$

$$f'(x) \leq L, \quad \forall x$$

Lo que se puede hacer con esto, es elegir puntos x e y cuya diferencia sea tal que $L|x - y|$ esté acotada por un ϵ dado, de modo que exista garantía de que los valores de la función estén incluidos dentro de ciertos rangos de variación. En este caso, se tendrá que $|f(x) - f(y)| \leq \epsilon$. De este modo, si dividimos el intervalo $[a, b]$ en una cantidad de puntos $a = x^0, x^1, \dots, x^p = b$ tales que $|x^i - x^{i+1}| \leq \epsilon/L$, tendremos que, en el intervalo $[x^i, x^{i+1}]$ se tiene que $|f(x) - f(x^i)| \leq \epsilon$.

En la Figura 1.3, por ejemplo, la función se ve acotada por las curvas verdes, cuya pendiente L o $-L$ tiene un valor absoluto superior a cualquier pendiente de la curva roja (mayor que cualquier derivada), lo que asegura que f nunca se saldrá de los márgenes, de modo que se puede tener certeza que el óptimo no se encuentra en partes más lejanas, de hecho, no se encuentra fuera del intervalo $[c, d]$.

De este modo, vamos encontrando “cajas” que “contienen” a f (al gráfico de f y esto permite definir un punto que tienen un error acotado con respecto al óptimo. Si hacemos la división del intervalo más fina, podremos tener una mucho mejor aproximación, de hecho. Esto ilustra cómo una propiedad adicional permite encontrar una solución aproximada tan precisa como se quiera. En efecto, dado que la solución estará en alguno de los intervalos, si queremos una solución ϵ -aproximada necesitamos que el ancho de los intervalos sea $\leq \epsilon/L$. Esto se logra dividiendo $[a, b]$ en p puntos equidistantes, con $p = L/\epsilon$.

Para encontrar la solución ϵ -aproximada tenemos que evaluar la función en todos los p puntos creados. En Optimización y en Computación se habla de la “complejidad” de un algoritmo. Esto es el número de

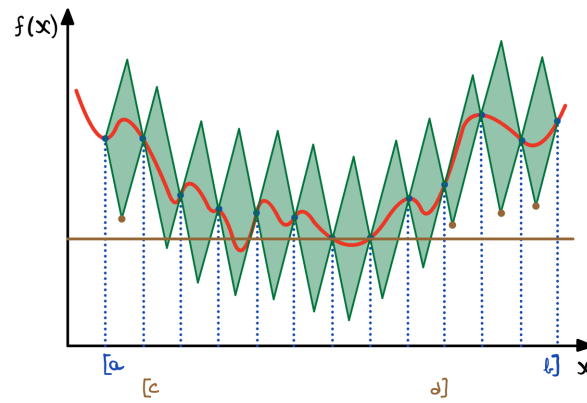


Figura 1.3: Uso de Lipschitz para descartar parte del dominio cuando se tiene un oráculo de orden cero

“iteraciones” (evaluaciones en este caso) necesarias para llegar a la meta (que es la aproximación, no la solución exacta). En este caso, esa complejidad es proporcional a L/ϵ . Esto refleja que si queremos más precisión, tenemos que trabajar más. Pero también indica que si la función no es muy “decente”, si L es muy grande, también tendremos que trabajar más.

Ahora bien, si tenemos una función de n variables, podemos extender esta idea. Lo que se debe hacer es dividir cada una de las coordenadas en intervalos de modo de forma una “grilla” en la que se debe evaluar la función. Es fácil darse cuenta que si usamos, digamos, p divisiones, entonces tendremos que hacer p^n evaluaciones de la función. La estimación de p es igual que en el caso de una dimensión, así que la complejidad, en este caso, es proporcional a $(L/\epsilon)^n$.

Podemos ver, entonces, que sólo evaluando valores de la función nuestra búsqueda de una aproximación al mínimo puede ser muy ineficiente. Acá surge, entonces, la necesidad de acceder a más información de f para construir algoritmos más inteligentes, por lo que deberemos acceder a oráculos de mayor orden. Adicionalmente se deben pedir ciertas estructuras en f más fuertes, por ejemplo, convexidad.

Capítulo 2

Métodos del Gradiente, Newton y extensiones

En este capítulo abordaremos el problema más simple de optimización: aproximar el mínimo de una función razonablemente “decente”, sin considerar restricciones de ningún tipo. Esto nos llevará a estudiar dos de los métodos clásicos más famosos, como son los métodos de Gradiente y Newton. Estudiaremos la forma cómo funcionan y también su comportamiento y convergencia. Estudiaremos también los métodos Quasi-Newton, que son extensiones que buscan ser más eficientes. Si bien todos estos métodos son antiguos, aún siguen siendo la piedra central para abordar muchos tipos de problemas.

2.1. La idea del Algoritmo de Descenso

El problema a estudiar para este caso es de la forma:

$$P) \min_{x \in \mathbb{R}^n} f(x)$$

Donde f es, al menos, continuamente diferenciable. La idea consiste en partir de un punto x_0 e ir moviéndose iterativamente a puntos donde el valor de la función objetivo disminuya progresivamente. Con esto en mente, se define:

Definición 2.1.1: Dirección de Descenso

$d \neq 0$ se dice dirección de descenso de f si existe $r > 0$ tal que:

$$f(x + \lambda d) < f(x), \quad \forall \lambda \geq 0, \lambda \leq r$$

Un ejemplo de esto se puede ver en la Figura 2.1, donde en el punto azul que está sobre la curva roja hay una dirección de descenso en $+y$.

Alternativamente, d es dirección de descenso si la derivada direccional en la dirección d (el producto punto de la dirección por el gradiente) es negativo:

$$d^T \nabla f(x) < 0$$

La idea general que surge de aquí es desarrollar un algoritmo que se mueva por direcciones de descenso. Esto es equivalente a lo que un excursionista haría bajando un cerro. Podríamos ir por un camino que valla hacia abajo, pero podría pasar que en algún momento dejemos de descender por ese camino específico. Entonces, es necesario cambiar de dirección. El algoritmo que se muestra a continuación hace eso: se mueve por una dirección de descenso lo más que se pueda, y después de eso pide una nueva dirección de descenso distinta a la actual.

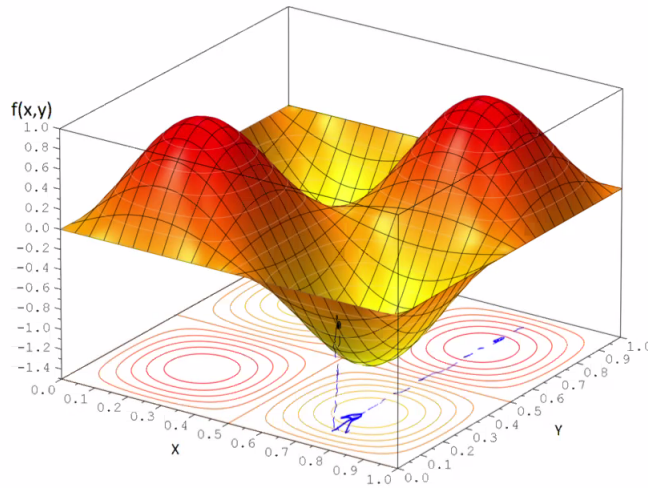


Figura 2.1: Dirección de descenso

Algoritmo del Método básico de Descenso

0. Sea $x^0 \in \mathbb{R}^n$, $k = 0$.
1. Sea d^k una dirección de descenso en x^k .
2. Verificar criterio parada: STOP si se cumple.
3. Sea λ^k solución óptima de:

$$\min_{\lambda \geq 0} f(x^k + \lambda d^k), \quad (\text{linesearch})$$

4. Sea $x^{k+1} = x^k + \lambda^k d^k$, $k \leftarrow k + 1$. Go to 1.

Normalmente un criterio de parada puede ser una cierta cantidad de iteraciones, un cierto tiempo de computo o, considerando que en el óptimo $\nabla f(x^*) = 0$ y que $\nabla f(x^k)_{k \rightarrow \infty} \rightarrow 0$, un punto donde $\|\nabla f(x)\| < \epsilon$, para un ϵ predeterminado. A su vez, el *linesearch* corresponde al paso de avance: cuánto avanzo en la dirección d^k , valor que se obtiene de un problema de optimización que naturalmente es más simple que el problema original dado que es de solo una variable. Por otro lado, el oráculo es el que da la dirección de descenso d dado el punto x^k en que se esté. Cabe mencionar que este tipo de algoritmo no garantiza que se encuentre el mínimo global, sino solo uno local. Para tener garantía de mínimo global, debe existir ciertas propiedades de convexidad.

Existen diversos métodos que usan este algoritmo básico, utilizando distintos oráculos y distintos enfoques. Algunos de estos son el Método del Gradiente con dirección de descenso $d^k = -\nabla f(x^k)$, el de Newton con $d^k = -[\nabla^2 f(x^k)]^{-1} \nabla f(x^k)$ y el de Quasi-Newton que tiene otro d^k .

2.2. Método del Gradiente

El Método del Gradiente o Método de Cauchy, utiliza un oráculo de primer orden (necesita las derivadas / gradientes de f), pues define la dirección como $d^k = -\nabla f(x^k)$. Esta dirección corresponde a la de máximo descenso desde un punto x^k .

Algoritmo del Método del Gradiente

0. Sea $x^0 \in \mathbb{R}^n$, $k = 0$ y $\epsilon > 0$
1. Dirección de descenso: $d^k = -\nabla f(x^k)$.
2. Si $\|\nabla f(x)\| < \epsilon$: STOP.
3. Sea λ^k solución de:

$$\min_{\lambda \geq 0} f(x^k + \lambda d^k)$$

con λ^k solución óptima de $\nabla f(x^k + \lambda^k d^k)^T d^k = 0$

4. Sea $x^{k+1} = x^k + \lambda^k d^k$, $k \leftarrow k + 1$. Go to 1.

La forma en que los puntos $x^0, x^1, x^2, \dots$ van evolucionando es tal que se va avanzando en direcciones ortogonales: $\nabla x^{k+1} \perp \nabla x^k$, $\forall k$. El que las direcciones de descenso sean ortogonales implica que $(d^k)^T (d^{k+1}) = 0$ y que por tanto se mueva en “zigzag”, tal como se ve en la Figura 2.2 para dos iteraciones.

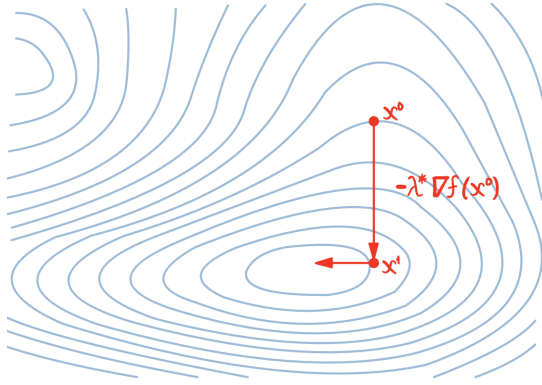


Figura 2.2: Dos iteraciones del Método del Gradiente

La prueba de que $\nabla x^{k+1} \perp \nabla x^k$ con $\nabla x^{k+1} = x^{k+1} - x^k = \lambda^k d^k$ se presenta a continuación. En el paso 3 del Método del Gradiente se encuentra el valor del λ mínimo resolviendo el problema:

$$\min_{\lambda \geq 0} f(x + \lambda d)$$

Cuya solución se encuentra derivando respecto a λ e igualando a cero:

$$\begin{aligned} \frac{\partial f(x + \lambda d)}{\partial \lambda} &= 0 \\ \nabla f(x + \lambda d)^T d &= 0 \end{aligned}$$

Si λ^{k-1} es la solución del problema de lineasearch correspondiente, entonces como

$$d^k = -\nabla f(x^k) = -\nabla f(x^{k-1} + \lambda^{k-1} d^{k-1})$$

tenemos que:

$$\begin{aligned} -\nabla f(x^{k-1} + \lambda^{k-1} d^{k-1})^T d^{k-1} &= 0 \Leftrightarrow -\nabla f(x^k)^T d^{k-1} = 0 \\ &\Leftrightarrow d^{kT} d^{k-1} = 0 \\ &\Leftrightarrow d^k \perp d^{k-1} \\ &\Leftrightarrow \lambda^k d^k \perp \lambda^{k-1} d^{k-1} \\ &\Leftrightarrow \nabla x^{k+1} \perp \nabla x^k \end{aligned}$$

Probándose que la dirección de descenso con el Método del Gradiente avanza de manera ortogonal en cada iteración.

Ejemplo:

Consideremos el problema:

$$\min f(x_1, x_2) = 5x_1^2 + x_2^2 + 4x_1x_2 - 14x_1 - 6x_2 + 20$$

cuya solución óptima es $(1, 1)$ con $f(1, 1) = 10$.

Para x^k dado podemos calcular todo:

$$d^k = -\nabla f(x_1^k, x_2^k) = \begin{pmatrix} -10x_1^k - 4x_2^k + 14 \\ -2x_2^k - 4x_1^k + 6 \end{pmatrix}$$

$$\begin{aligned} f(x^k + \lambda d^k) &= 5(x_1^k + \lambda d_1^k)^2 + (x_2^k + \lambda d_2^k)^2 + \\ &4(x_1^k + \lambda d_1^k)(x_2^k + \lambda d_2^k) - \\ &14(x_1^k + \lambda d_1^k) - 6(x_2^k + \lambda d_2^k) + 20 \end{aligned}$$

Luego:

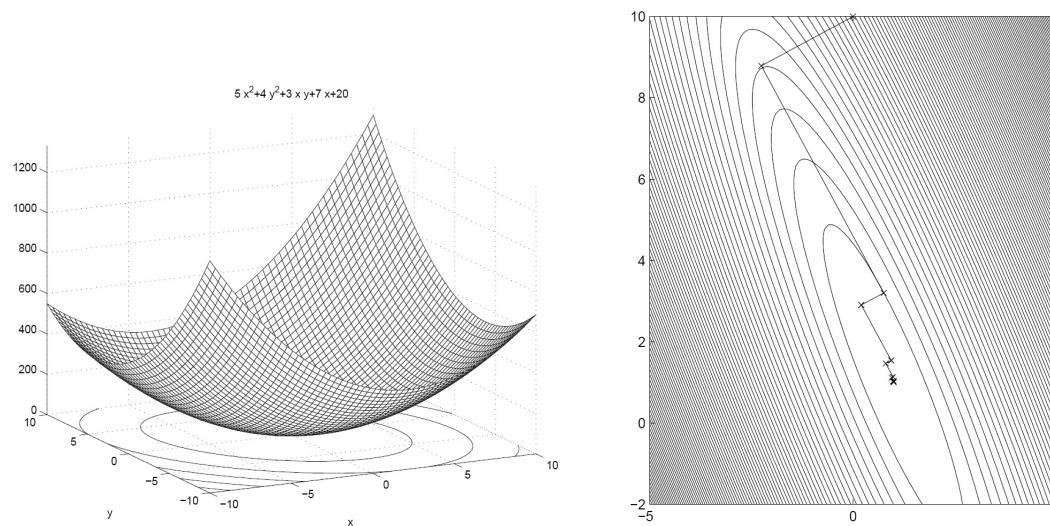
$$\lambda^k = \frac{(d_1^k)^2 + (d_2^k)^2}{2(5(d_1^k)^2 + (d_2^k)^2 + 4(d_1^k)(d_2^k))}$$

La siguiente tabla muestra los resultados, partiendo de $x^0 = (0, 10)$, con una tolerancia de $\epsilon = 10^{-6}$.

k	x_1^k	x_2^k	d_1^k	d_2^k	$\ d^k\ _2$	λ^k	$f(x^k)$
1	0.000000	10.000000	-26.000000	-14.000000	29.52964612	0.0866	60.000000
2	-2.252782	8.786963	1.379968	-2.562798	2.91071234	2.1800	22.222576
3	0.755548	3.200064	-6.355739	-3.422321	7.21856659	0.0866	12.987827
4	0.204852	2.903535	0.337335	-0.626480	0.71152803	2.1800	10.730379
5	0.940243	1.537809	-1.553670	-0.836592	1.76458951	0.0866	10.178542
6	0.805625	1.465322	0.082462	-0.153144	0.17393410	2.1800	10.043645
7	0.985392	1.131468	-0.379797	-0.204506	0.43135657	0.0866	10.010669
8	0.952485	1.113749	0.020158	-0.037436	0.04251845	2.1800	10.002608
9	0.996429	1.032138	-0.092842	-0.049992	0.10544577	0.0866	10.000638
10	0.988385	1.027806	0.004928	-0.009151	0.01039370	2.1800	10.000156

k	x_1^k	x_2^k	d_1^k	d_2^k	$\ d^k\ _2$	λ^k	$f(x^k)$
11	0.999127	1.007856	-0.022695	-0.012221	0.02577638	0.0866	10.000038
12	0.997161	1.006797	0.001205	-0.002237	0.00254076	2.1800	10.000009
13	0.999787	1.001920	-0.005548	-0.002987	0.00630107	0.0866	10.000002
14	0.999306	1.001662	0.000294	-0.000547	0.00062109	2.1800	10.000001
15	0.999948	1.000469	-0.001356	-0.000730	0.00154031	0.0866	10.000000
16	0.999830	1.000406	0.000072	-0.000134	0.00015183	2.1800	10.000000
17	0.999987	1.000115	-0.000332	-0.000179	0.00037653	0.0866	10.000000
18	0.999959	1.000099	0.000018	-0.000033	0.00003711	2.1800	10.000000
19	0.999997	1.000028	-0.000081	-0.000044	0.00009204	0.0866	10.000000
20	0.999990	1.000024	0.000004	-0.000008	0.00000907	2.1803	10.000000
21	0.999999	1.000007	-0.000020	-0.000011	0.00002250	0.0866	10.000000
22	0.999998	1.000006	0.000001	-0.000002	0.00000222	2.1817	10.000000
23	1.000000	1.000002	-0.000005	-0.000003	0.00000550	0.0866	10.000000
24	0.999999	1.000001	0.000000	-0.000000	0.00000054	0.0000	10.000000

A continuación se muestra un gráfico de f y un mapa de las curvas de nivel, mostrando las iteraciones. Se puede ver como el avance se hace en direcciones ortogonales.



Hacemos notar finalmente que, en términos de trabajo, el método es “barato”: en cada iteración sólo requiere una evaluación del gradiente y resolver el problema de la búsqueda unidireccional, pero ya dijimos que en la práctica eso puede hacerse en forma aproximada y muy rápida.

A continuación se muestra un código Python que implemente el Método del Gradiente para una función de la forma:

$$f(x) = \frac{1}{2}x^T Qx + c^T x + \alpha(5 - x_n)^4$$

```
import numpy as np
import scipy.optimize
from apendice import generar_datos, subrutina, funcion

def gradiente(Q, c, alpha, x0, epsilon, iteracion_maxima):
    """
    Esta funcion es una aplicacion del metodo del gradiente. Su entrada posee:
    - Q, c y alpha: parametros de la funcion definida.
    - x0: punto inicial de prueba.
    - epsilon: error/ tolerancia deseada.
    - iteracion_maxima: numero maximo de iteraciones.
    Su retorno es:
    - valor: valor de la funcion evaluada en x en la iteracion actual.
    - x: solucion en la que se alcanza el valor objetivo.
    """
    # 0: Se definen los parametros iniciales
    iteracion, x = 0, x0
    m, n = Q.shape

    while iteracion <= iteracion_maxima:
        # 1: Se obtiene la direccion de descenso
        [valor, gradiente] = subrutina(x, Q, c, alpha, m, n, "gradiente")
        direccion_descenso = - gradiente

        # 2: Se analiza el criterio de parada segun la norma
        if np.linalg.norm(gradiente, ord=2) <= epsilon: break

        # 3: Se resuelve el subproblema de lambda
        lambda_ = scipy.optimize.fminbound(
            funcion, 0, 10, args=(Q, c, x, alpha, direccion_descenso, n))

        # 4: Se actualiza el valor de x para la siguiente iteracion del algoritmo
```

```

    x, iteracion = x + lambda_ * direccion_descenso, iteracion + 1

    return valor, x

if __name__ == "__main__":
    np.random.seed(2121)
    n = 30
    alpha, iteracion_maxima, epsilon, x0 = 10, 1000, 0.001, np.ones((n, 1))
    Q, c = generar_datos(n)

    valor, x = gradiente(Q, c, alpha, x0, epsilon, iteracion_maxima)

```

2.3. Método de Newton

Por otro lado, está el Método de Newton, que utiliza un oráculo de segundo orden, pues define la dirección de descenso como el negativo de la inversa del Hessiano multiplicado por el gradiente: $d^k = -[\nabla^2 f(x^k)]^{-1} \nabla f(x^k)$, con $\nabla^2 f(x)$ definida positivo.

Vamos a explicar de dónde viene esta expresión y lo haremos primera para una variable. La idea central es aproximar el comportamiento de la función $f(y)$ usando el Polinomio de Taylor de segundo orden:

$$f(y) \approx f(x) + f'(x)(y - x) + \frac{1}{2}f''(x)(y - x)^2$$

Se puede encontrar el mínimo de esta aproximación derivando e igualando a cero, considerando que x es constante e y es la variable:

$$\begin{aligned} \frac{df(y)}{dy} = 0 &\rightarrow \frac{d}{dy} \left(f(x) + f'(x)(y - x) + \frac{1}{2}f''(x)(y - x)^2 \right) = 0 \\ f'(x) + f''(x)(y - x) &= 0 \\ y &= x - \frac{f'(x)}{f''(x)} \end{aligned}$$

De este modo, dado un punto inicial x^0 , obtenemos un segundo punto x^1 como aquel que es el óptimo de la aproximación de segundo orden de $f(x^0)$, de modo que: $x^1 = x^0 - f'(x^0)/f''(x^0)$. Ahora repetimos la minimización de la aproximación, pero en x^1 y el punto siguiente x^2 se obtiene como: $x^2 = x^1 - f'(x^1)/f''(x^1)$. En general, la iteración de puntos es de la forma:

$$x^{k+1} = x^k - \frac{f'(x^k)}{f''(x^k)}$$

Una intuición de la evolución de los puntos es la de la Figura 2.3

De manera análoga, para el caso de una función de varias variables, también usamos el polinomio de Taylor, pero todo debe manejarse en forma vectorial y matricial. La aproximación de segundo orden de una función f en x es:

$$q(y) = f(x) + \nabla f(x)^T (y - x) + \frac{1}{2}(y - x)^T \nabla^2 f(x) (y - x)$$

Para encontrar el mínimo de esta función se saca su gradiente y se iguala al vector cero:

$$\begin{aligned} \nabla q(y) = 0 &\rightarrow \nabla \left(f(x) + \nabla f(x)^T (y - x) + \frac{1}{2}(y - x)^T \nabla^2 f(x) (y - x) \right) = 0 \\ \nabla f(x) + \nabla^2 f(x)(y - x) &= 0 \\ y &= x - [\nabla^2 f(x)]^{-1} \nabla f(x) \end{aligned}$$

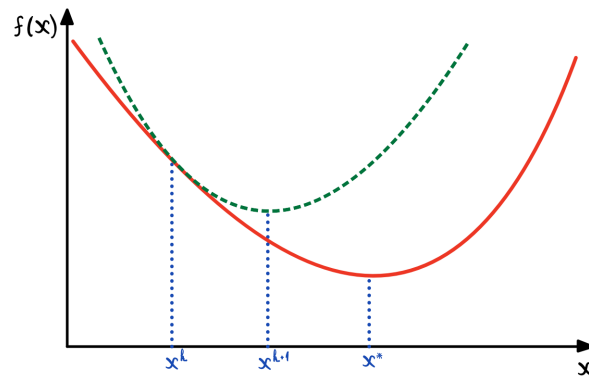


Figura 2.3: Avance con el Método de Newton dado la aproximación cuadrática de la función

El valor de y corresponde a el vector que minimiza la aproximación cuadrática de f evaluado en el vector x . Notemos ahora que para que el método tenga sentido como método de descenso, es necesario que la dirección cumpla con ser dirección de descenso, es decir:

$$\nabla f(x^k)^T d < 0$$

En el caso de Newton, como se tiene que $d^k = -[\nabla^2 f(x^k)]^{-1} \nabla f(x^k)$, entonces se debe cumplir que:

$$\begin{aligned} -\nabla f(x^k)^T [\nabla^2 f(x^k)]^{-1} \nabla f(x^k) &< 0 \\ \nabla f(x^k)^T [\nabla^2 f(x^k)]^{-1} \nabla f(x^k) &> 0 \end{aligned}$$

Esto se cumple si $[\nabla^2 f(x^k)]^{-1}$ es una matriz definida positiva y, por lo tanto, $\nabla^2 f(x^k)$, el Hessiano, también debe ser definida positiva. Esto se logra, efectivamente, si $f(x)$ es globalmente convexa.

El siguiente es un pseudo-código del algoritmo:

Algoritmo del Método de Newton

0. Sea $x^0 \in \mathbb{R}^n$, $k = 0$ y $\epsilon > 0$
1. Dirección de descenso: $d^k = -[\nabla^2 f(x^k)]^{-1} \nabla f(x^k)$, con $\nabla^2 f(x)$ definida positivo.
2. Si $\|\nabla f(x^k)\| \leq \epsilon$: STOP.
3. Sea λ^k solución de:

$$\min_{\lambda \geq 0} f(x^k + \lambda^k d^k)$$

con λ^k solución óptima de $\nabla f(x^k + \lambda^k d^k)^T d^k = 0$

4. Sea $x^{k+1} = x^k + \lambda^k d^k$, $k \leftarrow k + 1$. Go to 1.

Si f es convexa, entonces Newton converge desde cualquier punto de partida al mínimo de la función. Y si $\nabla^2 f(x)$ es definida positiva entonces garantiza convexidad estricta local.

El siguiente es un código Python que implementa el Método de Newton. Nótese que la única diferencia con respecto al que implementa el Método del Gradiente es el punto en donde se elige la dirección de descenso.

```
import numpy as np
import scipy.optimize
from apendice import generar_datos, subrutina, funcion
```

```
def newton(Q, c, alpha, x0, epsilon, iteracion_maxima):
    """
    Esta funcion es una aplicacion del metodo de Newton. Su entrada posee:
    - Q, c, alpha: parametros de la funcion definida.
```

```

- x0: punto inicial de prueba.
- epsilon: error/ tolerancia deseada.
- iteracion_maxima: numero maximo de iteraciones.
Su retorno es:
- valor: valor de la funcion evaluada en x en la iteracion actual.
- x: solucion en la que se alcanza el valor objetivo.
"""
# 0: Se definen los parametros iniciales
iteracion, x = 0, x0
m, n = Q.shape

while iteracion <= iteracion_maxima:
    # 1: Se obtiene direccion de descenso
    [valor, gradiente, hessiano] = subrutina(x, Q, c, alpha, m, n, "newton")
    direccion_descenso = np.dot(-np.linalg.inv(hessiano), gradiente)

    # 2: Se analiza el criterio de parada segun la norma
    if np.linalg.norm(gradiente, ord=2) <= epsilon: break

    # 3: Se resuelve el subproblema de lambda
    lambda_ = scipy.optimize.fminbound(
        funcion, 0, 10, args=(Q, c, x, alpha, direccion_descenso, n))

    # 4: Se actualiza el valor de x para la siguiente iteracion del algoritmo
    x, iteracion = x + lambda_ * direccion_descenso, iteracion + 1

return valor, x

if __name__ == "__main__":
    np.random.seed(2121)
    n = 30
    alpha, iteracion_maxima, epsilon, x0 = 10, 1000, 0.001, np.ones((n, 1))
    Q, c = generar_datos(n)

    valor, x = newton(Q, c, alpha, x0, epsilon, iteracion_maxima)

```

Cápsula de video 2.3.1: Los Métodos de Descenso

Profesor Jorge Vera explica en clases lo que está detrás de los métodos de descenso, en particular del Método del Gradiente y el Método de Newton:
https://youtu.be/Eow_2jmRU6g

2.4. Midiendo la eficiencia de un algoritmo

Una de los temas que más nos interesaran en este curso es la eficiencia computacional de los distintos algoritmos. Para esto, necesitaremos estimar de qué forma las distintas iteraciones se van acercando a la meta. En esta sección introduciremos algunos elementos de la medición de convergencia, que se conocen como “tasas de convergencia”. Posteriormente veremos cómo se comportan los métodos que ya hemos comentado.

Supongamos que un algoritmo ha generado puntos $x^0, x^1, \dots, x^k$ a lo largo de las iteraciones, (que se puede expresar como $\{x^k\}$). Vamos a suponer que esta sucesión efectivamente se acerca a la solución óptima x^* . Entonces, la diferencia entre la sucesión $\{x^k\}$ y x^* disminuye a medida que se itera más (k aumenta), es decir, converge a 0.

Definición 2.4.1: Convergencia Lineal

Se dice que la sucesión $\{x^k\}$ converge a x^* en forma lineal si

$$\|x^{k+1} - x^*\|_2 \leq \alpha \|x^k - x^*\|_2, \quad k = 1, 2, \dots$$

Donde $0 < \alpha < 1$.

El valor de α depende las propiedades de la función. Esto significa que el error que se tiene en la iteración $k + 1$ es menor a una proporción α del error que se tenía en la iteración k .

Definición 2.4.2: Convergencia Cuadrática

Decimos que la sucesión $\{x^k\}$ converge a x^* en forma cuadrática si

$$\|x^{k+1} - x^*\|_2 \leq \beta \|x^k - x^*\|_2^2, \quad k = 1, 2, \dots$$

para una constante $\beta > 0$.

Nótese que como la sucesión $\{x^k\}$ converge a x^* , entonces $\|x^k - x^*\|_2$ converge a 0, por lo que al estar el término de error al cuadrado es más pequeño y así la diferencia de las soluciones respecto a x^* converge más rápidamente que con la convergencia lineal. La convergencia cuadrática es más rápida que la lineal, en el sentido que requiere menos iteraciones para llegar a la misma meta de error, como mostraremos ahora.

Se puede determinar una cota mínima para el número de iteraciones k que se deben hacer para llegar a un determinado error $\epsilon > 0$. En el caso de la convergencia lineal, este valor se puede calcular del siguiente modo: Sea $e_k = \|x^k - x^*\|_2$, luego por definición de la convergencia lineal:

$$e_k \leq \alpha e_{k-1}, \quad k \in \mathbb{Z}_0^+$$

Desarrollándose múltiples veces la expresión anterior se obtiene:

$$\begin{aligned} e_k &\leq \alpha [\alpha e_{k-2}] \\ &\leq \alpha^2 e_{k-2} \\ &\leq \alpha^3 e_{k-3} \\ &\vdots \\ &\leq \alpha^k e_0 \end{aligned}$$

Donde $e_0 = \|x^0 - x^*\|_2$. Luego, para que $e_k \leq \epsilon$, basta pedir que $\alpha^k e_0 \leq \epsilon$. Esto quiere decir que la cantidad de k iteraciones que se deben realizar para alcanzar dicha convergencia es de:

$$\begin{aligned} \alpha^k e_0 &\leq \epsilon \\ \alpha^k &\leq \frac{\epsilon}{e_0} \\ k \ln(\alpha) &\leq \ln\left(\frac{\epsilon}{e_0}\right) \\ k &\geq \frac{1}{|\ln(1/\alpha)|} \ln\left(\frac{e_0}{\epsilon}\right) \end{aligned}$$

Lo que importa acá es ver como depende del ϵ , tal como se ve, la cantidad de iteraciones que se deben realizar es de orden de $\log(1/\epsilon)$, lo cual se denota de la siguiente forma:

$$\mathcal{O}\left(\log\left(\frac{1}{\epsilon}\right)\right)$$

Esto corresponde a la complejidad de la convergencia lineal. Mientras más pequeño es ϵ , más iteraciones se deben hacer como era de esperarse.

Para el caso de la convergencia cuadrática, el valor de la cota mínima de iteraciones k que se deben hacer para alcanzar un error máximo de ϵ se calcula de modo análogo: Sea $e_k = \|x^k - x^*\|_2$, luego por definición de la convergencia cuadrática, podemos plantear la siguiente desigualdad:

$$e_k \leq \beta e_{k-1}^2, \quad k \in \mathbb{Z}_0^+$$

Desarrollando múltiples veces la expresión anterior se obtiene:

$$\begin{aligned} e_k &\leq \beta e_{k-1}^2 \\ &\leq \beta [\beta e_{k-2}^2]^2 \\ &\leq \beta^3 e_{k-2}^4 \\ &\vdots \\ &\leq \beta^{2^k - 1} e_0^{2^k} \end{aligned}$$

Nos interesa saber cuántas iteraciones debemos realizar para asegurar un error $\epsilon \geq 0$, es decir:

$$\begin{aligned} \beta^{2^k - 1} e_0^{2^k} &\leq \epsilon \\ (2^k - 1) \ln(\beta) + 2^k \ln(e_0) &\leq \ln(\epsilon) \\ 2^k \ln(\beta) + 2^k \ln(e_0) - \ln(\beta) &\leq \ln(\epsilon) \\ 2^k [\ln(\beta) + \ln(e_0)] &\leq \ln(\beta) + \ln(\epsilon) \\ 2^k \ln(\beta e_0) &\leq \ln(\beta \epsilon) \end{aligned}$$

Como $e_0 > \epsilon$, podemos plantear tres escenarios. Los dos primeros son escenarios que no resultan posibles:

$$\begin{array}{ll} (1) & \beta e_0 > 1 \wedge \beta \epsilon > 1 : \\ & 2^k \leq \frac{\ln(\beta \epsilon)}{\ln(\beta e_0)} < 1 \Rightarrow k < 0 \\ & \text{que es una contradicción, ya que } k \in \mathbb{Z}_0^+. \\ (2) & \beta e_0 > 1 \wedge \beta \epsilon < 1 : \\ & 2^k \leq \frac{\ln(\beta \epsilon)}{\ln(\beta e_0)} < 0 \\ & \text{lo cual es una contradicción.} \end{array}$$

El tercer escenario sí es posible:

$$(3) \quad \beta e_0 < 1 \wedge \beta \epsilon < 1: {}^1$$

$$\begin{aligned} 2^k &\geq \frac{\ln(\beta \epsilon)}{\ln(\beta e_0)} > 1 \\ k &\geq \frac{1}{\ln(2)} \ln \left(\frac{\ln(\beta \epsilon)}{-|\ln(\beta e_0)|} \right) \\ k &\geq \frac{1}{\ln(2)} \ln \left(\frac{\ln(1/\beta \epsilon)}{|\ln(\beta e_0)|} \right) \end{aligned}$$

De este modo, la cantidad de iteraciones que se deben realizar en el caso de un algoritmo de convergencia cuadrática es del orden de:

$$\mathcal{O} \left(\log \left(\log \left(\frac{1}{\epsilon} \right) \right) \right)$$

¹Notar que como se sabe que $\ln(\beta e_0)$ es negativo, entonces: $\ln(\beta e_0) = -|\ln(\beta e_0)|$. Esto se utiliza en el segundo cálculo.

Cápsula de video 2.4.1: Tasas de convergencia

Profesor Jorge Vera explica en clases los detalles y el significado de las tasas de convergencia lineal y cuadráticas:

https://youtu.be/DR_s9HCK15Y

2.4.1. Convergencia del Método del Gradiente

En esta sección vamos a ilustrar que el Método del Gradiente tiene convergencia lineal. Esto lo haremos para un problema cuadrático estándar:

$$\min_{x \in \mathbb{R}^n} f(x) = \frac{1}{2} x^T Q x + c^T x,$$

donde Q es una matriz definida positiva y simétrica ($Q = Q^T$). Vamos a ilustrar lo que ocurre con dos casos particulares de esta función:

Caso 1:

$$Q_1 = \begin{pmatrix} 20 & 5 \\ 5 & 1 \end{pmatrix}, c = \begin{pmatrix} 14 \\ 6 \end{pmatrix}$$

Caso 2:

$$Q_2 = \begin{pmatrix} 20 & 5 \\ 5 & 16 \end{pmatrix}, c = \begin{pmatrix} 14 \\ 6 \end{pmatrix}$$

A continuación mostramos tablas que indican las sucesivas iteraciones del método y la convergencia:

Para el Caso 1:

k	x_1^k	x_2^k	$\ d^k\ _2$	λ^k	$f(x^k)$	$\frac{f(x^k) - f(x^*)}{f(x^{k-1}) - f(x^*)}$
1	40.000000	-100.000000	286.06293014	0.0506	6050.000000	
2	25.542693	-99.696700	77.69702948	0.4509	3981.695128	0.658079
3	26.277558	-64.668130	188.25191488	0.0506	2620.587793	0.658079
4	16.763512	-64.468535	51.13075844	0.4509	1724.872077	0.658079
5	17.247111	-41.416980	123.88457127	0.0506	1135.420663	0.658079
6	10.986120	-41.285630	33.64806192	0.4509	747.515255	0.658079
7	11.304366	-26.115894	81.52579489	0.0506	492.242977	0.658079
8	7.184142	-26.029455	22.14307211	0.4509	324.253734	0.658079
9	7.393573	-16.046575	53.65038732	0.0506	213.703595	0.658079
10	4.682141	-15.989692	14.57188362	0.4509	140.952906	0.658079

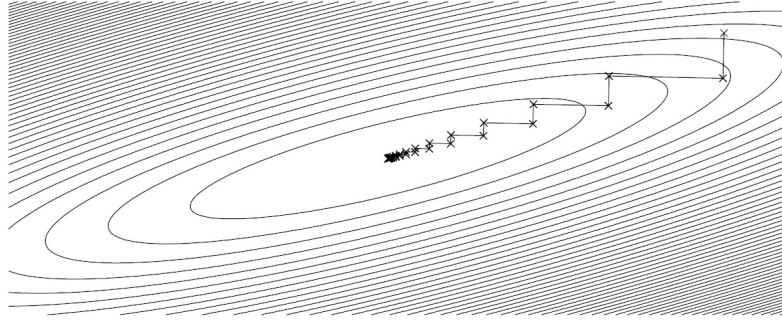
k	x_1^k	x_2^k	$\ d^k\ _2$	λ^k	$f(x^k)$	$\frac{f(x^k) - f(x^*)}{f(x^{k-1}) - f(x^*)}$
20	0.460997	0.948466	1.79847660	0.4509	3.066216	0.658079
30	-0.059980	3.038991	0.22196980	0.4509	0.965823	0.658079
40	-0.124280	3.297005	0.02739574	0.4509	0.933828	0.658079
50	-0.132216	3.328850	0.00338121	0.4509	0.933341	0.658079
60	-0.133195	3.332780	0.00041731	0.4509	0.933333	0.658078
70	-0.133316	3.333265	0.00005151	0.4509	0.933333	0.658025
80	-0.133331	3.333325	0.00000636	0.4509	0.933333	0.654656
90	-0.133333	3.333332	0.00000078	0.0000	0.933333	0.000000

Para el Caso 2:

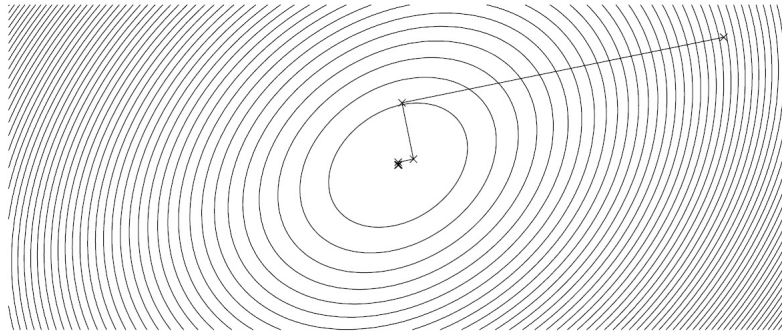
k	x_1^k	x_2^k	$\ d^k\ _2$	λ^k	$f(x^k)$	$\frac{f(x^k) - f(x^*)}{f(x^{k-1}) - f(x^*)}$
1	40.000000	-100.000000	1434.79336491	0.0704	76050.000000	
2	19.867118	-1.025060	385.96252652	0.0459	3591.615327	0.047166
3	2.513241	-4.555081	67.67315150	0.0704	174.058930	0.047166
4	1.563658	0.113150	18.20422450	0.0459	12.867208	0.047166
5	0.745149	-0.053347	3.19185713	0.0704	5.264475	0.047166
6	0.700361	0.166834	0.85861649	0.0459	4.905886	0.047166
7	0.661755	0.158981	0.15054644	0.0704	4.888973	0.047166
8	0.659643	0.169366	0.04049732	0.0459	4.888175	0.047166
9	0.657822	0.168996	0.00710064	0.0704	4.888137	0.047166
10	0.657722	0.169486	0.00191009	0.0459	4.888136	0.047166
11	0.657636	0.169468	0.00033491	0.0704	4.888136	0.047166
12	0.657632	0.169491	0.00009009	0.0459	4.888136	0.047161
13	0.657628	0.169490	0.00001580	0.0704	4.888136	0.047068
14	0.657627	0.169492	0.00000425	0.0459	4.888136	0.045002
15	0.657627	0.169491	0.00000075	0.0000	4.888136	0.000000

Podemos observar que en el caso 1 la convergencia es significativamente más lenta. Para entender esto es interesante observar cómo son las curvas de nivel de la función objetivo. Estas se muestran en las siguientes figuras:

Para el Caso 1:



Para el Caso 2:



Podemos observar que en el caso 1, las curvas de nivel son mucho más “excéntricas” que en el caso 2. Eso hace que, dado que el método avanza en direcciones ortogonales, se produzca un significativo “zigzag” a medida que avanzan las iteraciones. La información respecto a la excentricidad está contenida en la matriz Q .

Sean μ_{max} y μ_{min} el mayor y menor valor propio de Q respectivamente. Sea

$$\kappa(Q) = \frac{\mu_{max}}{\mu_{min}}$$

Este valor se conoce como el “número de condicionamiento” de la matriz Q .

Teorema 2.4.1: Convergencia lineal del Método del Gradiente

Si Q es definida positiva, entonces

$$f(x^{k+1}) - f(x^*) \leq \left(\frac{\kappa(Q) - 1}{\kappa(Q) + 1} \right)^2 (f(x^k) - f(x^*)), k = 1, 2, \dots$$

Mientras mayor sea $\kappa(Q)$ más excéntricas son las curvas de nivel, y la convergencia del algoritmo es más lenta. En el ejemplo tenemos que para el caso 1, $\kappa(Q_1) = 90,1$ mientras que para el caso 2, $\kappa(Q_2) = 1,85$ y eso se refleja en las diferencias de convergencia. De hecho, si las curvas de nivel fueran círculos perfectos, la convergencia es inmediata en una iteración ya que todas las direcciones $-\nabla f(x)$ apuntan a centro común de las curvas de nivel, que es la solución óptima.

Dado $\epsilon > 0$, podemos seguir el desarrollo del comienzo de la sección y concluir que el número de iteraciones necesario para llegar a ese error es

$$k = O\left(\frac{1}{\log(\alpha)} \log\left(\frac{1}{\epsilon}\right)\right), \alpha = \frac{\kappa(Q) + 1}{\kappa(Q) - 1}$$

El análisis que hemos presentado acá es para el caso de una función cuadrática, pero puede extenderse a una función convexa cualquiera. En efecto, si f es convexa y x^* es su mínimo global, ya sabemos que una función puede aproximarse mediante su polinomi de Taylor de orden 2. Esa aproximación suele ser muy buena en el caso de funciones cuadráticas, así que tendremos que, en torno a x^* :

$$f(y) \approx f(x^*) + \nabla f(x^*)^T (y - x^*) + \frac{1}{2} (y - x^*)^T \nabla^2 f(x^*) (y - x^*)$$

Entonces, si el punto inicial x^0 está suficientemente cerca de x^* podemos pensar que el problema de minimizar f es muy similar al de minimizar su aproximación cuadrática. Entonces, en este caso, la velocidad de convergencia estará gobernada por las propiedades de $\nabla^2 f(x^*)$, más específicamente, por su condicionamiento. Se puede probar formalmente que este argumento es correcto pero ese desarrollo escapa del nivel de este curso.

2.4.2. Cambios de variable para mejorar el condicionamiento

Existen formas que permiten solucionar (aunque sea parcialmente) el problema de la fuerte dependencia de la eficiencia del método del gradiente en el condicionamiento del Hessiano. Aquí ilustramos una de esas formas, que es mediante un cambio de variables.

Sea $f(x) = \frac{1}{2} x^T Q x + b^T x$ donde Q es simétrica y definida positiva. Consideremos un cambio de variable de la forma $x = Ay$, donde $A \in \mathbb{R}^{n \times n}$ asumiremos que es invertible. Sea $g(y) = f(Ay)$. La función transformada por el cambio de variable es:

$$g(y) = \frac{1}{2} (Ay)^T Q (Ay) + b^T (Ay) = \frac{1}{2} y^T A^T Q A y + b^T A y$$

Dado que Q es una matriz simétrica y definida positiva, podemos utilizar su diagonalización y reemplazar en la expresión anterior por $Q = B^T D B$:

$$g(y) = \frac{1}{2} y^T A^T (B^T D B) A y + b^T A y$$

Sea $\bar{Q} = A^T (B^T D B) A$, notemos que si $\bar{Q}$ es una matriz diagonal, obtenemos una función cuadrática con un término lineal adicional que solamente desplaza y/o amplía la curva. Por lo tanto, queremos que $\bar{Q}$ sea diagonal. Para que se cumpla lo anterior, $BA = I$, es decir, $A = B^{-1}$. Estos nos lleva a la siguiente expresión cuadrática:

$$g(y) = \frac{1}{2} y^T D y + b^T B^{-1} y$$

Ahora, buscamos una transformación que nos lleve a la forma cuadrática $\frac{1}{2}z^T z$. Para ello podemos aplicar el cambio de variable, $y = D^{-1/2}z$. Finalmente, combinando ambas transformaciones, obtenemos que $A = (BD^{1/2})^{-1}$. Dicha transformación garantiza que el método del gradiente converja en una iteración. Por supuesto, tenemos que diagonalizar Q para lograr esto, lo cual es un proceso numérico bastante intensivo. Existen métodos más económicos que obtienen parcialmente información de la diagonalización de Q y permiten construir una matriz adecuada para el cambio de variable e igual mejorar significativamente en comportamiento de método, pero no los estudiaremos aquí. Todo este proceso es muy importante en optimización numérica y se suele llamar “precondicionamiento” del problema.

2.4.3. Velocidad de convergencia para el Método de Newton

En cuanto a la convergencia en el Método de Newton, existe el siguiente teorema:

Teorema 2.4.2: Convergencia del Método de Newton

Sea f dos veces continuamente diferenciable, x^* un mínimo local de f y tal que $\nabla^2 f(x^*)$ sea definida positiva. Supongamos además que existen $r > 0$ y $L > 0$ tales que la variación de los hessianos en el entorno de x^* se encuentra acotado:

$$\|\nabla^2 f(x) - \nabla^2 f(y)\| \leq L\|x - y\|, \quad \forall x, y \in B(x^*, r)$$

Entonces, existe $\gamma > 0$ tal que si $\|x^0 - x^*\| < \gamma$, tal que la sucesión $\{x^k\}_k$ generada por el método converge a **velocidad cuadrática**, es decir, existe $\beta > 0$ tal que:

$$\|x^{k+1} - x^*\| \leq \beta \|x^k - x^*\|^2, \quad \forall k = 0, 1, \dots$$

Si f es convexa, y por lo tanto tiene mínimo, entonces Newton converge desde cualquier punto de partida al mínimo de f . Si el Hessiano $\nabla^2 f(x)$, es definida positiva para todo x , entonces la función es convexa y se garantiza lo anterior. Si por lo menos en el óptimo x^* , $\nabla^2 f(x^*)$ es definida positiva, entonces al menos se garantiza “convexidad estricta local” ya que la función es convexa en el entorno de x^* .

Una de las condiciones que pide el teorema acota la norma de la variación de los Hessianos de f , proporcional a la diferencia de los puntos donde se evalúan los Hessianos:

$$\|\nabla^2 f(x) - \nabla^2 f(y)\| \leq L\|x - y\|, \quad \forall x, y \in B(x^*, r),$$

lo que es una condición de Lipschitz local para $\nabla^2 f$. Si L es pequeño, entonces la variación de la curvatura en f es pequeña. Si L es demasiado grande entonces γ tiene que ser demasiado pequeño y, por lo tanto, $\|x^0 - x^*\|$ es muy pequeño y entonces tendremos que partir muy cerca del óptimo para que Newton funcione. De este modo, cuando f tiene un comportamiento muy caótico, se debe partir iterando cerca del óptimo para garantizar la convergencia. Por otro lado, $\|\nabla^2 f(x^*)^{-1}\|_2$, al igual que L , influye en el valor de β y γ , pues debe estar acotada para garantizar la convergencia cuadrática. Esto ya que, en caso contrario, en el entorno del óptimo f será muy plana y Newton no funciona tan bien. Ejemplo de esto último es la función de una variable $f(x) = x^4$. Es fácil deducir que f es perfectamente convexa y el mínimo es $x = 0$, pero $f''(0) = 0$ (el Hessiano NO es definida positiva) y la convergencia al óptimo es sólo lineal, no cuadrática.

Podemos tratar de comparar el comportamiento del Método de Newton con el del Método del Gradiente, a partir de estos resultados teóricos de convergencia. Como sabemos, el método del Gradiente se ve afectado por la excentricidad de las curvas de nivel de la función cerca del óptimo. Esa excentricidad se puede medir con el siguiente valor:

$$\kappa = \|\nabla^2 f(x^*)\|_2 \|\nabla^2 f(x^*)^{-1}\|_2$$

Siempre podemos normalizar el problema de modo tal que $\|\nabla^2 f(x^*)\| = 1$, así que, entonces:

$$\kappa = \|\nabla^2 f(x^*)^{-1}\|_2 = \frac{1}{h}$$

Si la excentricidad es grande, es decir, κ es grande, entonces h será pequeño. Esto implica que $\|x^0 - x^*\|$ también será pequeño, para un L fijo. Esto implica que β tenderá a crecer, y al crecer, el impacto de reducción

de error en la convergencia cuadrática se ve, obviamente, afectado. Sin embargo, observemos la relación de la convergencia:

$$\|x^{k+1} - x^*\| \leq \beta \|x^k - x^*\|^2, \quad \forall k = 0, 1, \dots$$

Podemos ver que, si aún β no es muy pequeño, el efecto de reducción de error introducido por el término cuadrático $\|x^k - x^*\|^2$ es mucho mayor, compensando cualquier aumento de β .

Por otro lado, en la convergencia lineal del método de gradiente, tenemos una de convergencia relación lineal, de la forma

$$f(x^{k+1}) - f(x^*) \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^2 (f(x^k) - f(x^*)), k = 1, 2, \dots$$

Si bien esta relación es para errores en la función objetivo, eso no impide que hagamos el argumento general. Además, la relación puede ser llevada a una en x , con algún “factor de escala” adecuado usando el Teorema del Valor Medio. Podemos ver de esta última relación que un aumento de κ impacta directamente en la tasa de reducción del error, que no tiene ninguna disminución adicional, como en el caso de la convergencia cuadrática. Esto explica, aproximadamente, el por qué el Método de Newton es menos sensible a la curvatura de la función.

2.4.4. Costo por iteración del Método del Gradiente y Newton

Hasta el momento lo que hemos analizado es el número de iteraciones que requiere un algoritmo para llegar a la meta, digamos, una solución ϵ -aproximada o ϵ -óptima. Sin embargo, esto no nos dice mucho respecto a cuánto tiempo real (tiempo de reloj) tardaremos en lograr esa meta. Esto es porque el costo en tiempo por iteración puede ser diferente según el método. Más en específico, el tiempo total para llegar a una meta es la cantidad de iteraciones multiplicado por el tiempo de iteración.

$$\text{Tiempo total} = n^\circ \text{ iteraciones} \times \text{tiempo por iteración}$$

El tiempo por iteración se estima en proporción a la cantidad de operaciones aritméticas de punto flotante que el procesador de nuestro computador debe realizar. Operaciones de punto flotante son las sumas, restas, multiplicaciones y divisiones. A esto es a lo que se llama “flop”, que es un acrónimo de “floating point operation”. En genral, se considera que la verdadera sobrecarga de tiempo viene de las multiplicaciones y divisiones, las sumas y restas son despreciables, en términos comparativos.

En lo que sigue estimaremos el costo por iteración, el flops, de cada uno de los métodos (gradiente y Newton), concentrándonos fundamentalmente en las multiplicaciones. En cada una de las iteraciones, cada uno de los métodos utiliza una forma distinta de calcular el punto siguiente (x^k) según donde se está (x^{k+1}). En el caso del Método del Gradiente este es:

$$x^{k+1} = x^k - \lambda \nabla f(x^k)$$

lo que implica una suma y una multiplicación por cada coordenada, y hay n de ellas. Esto nos da un esfuerzo de $\mathcal{O}(n)$ “flops”. Es importante hacer notar que aquí no estamos contando el detalle de las operaciones necesarias para evaluar $\nabla f(x)$, estamos suponiendo que es un valor constante en términos de tiempo, de decir, es $\mathcal{O}(1)$. Un análisis más completo debería considerarlo pero eso dependerá de cada problema particular. En Ciencias de la Computación se dice que estamos usando un “modelo de caja negra” donde la caja negra es el oráculo que nos entrega el valor del gradiente, en tiempo fijo.

Con Newton la dirección es diferente y tenemos que invertir el Hessiano y multiplicarlo por el gradiente, pues el punto siguiente en una iteración es:

$$x^{k+1} = x^k - \lambda [\nabla^2 f(x^k)]^{-1} \nabla f(x^k)$$

lo que implica que se debe invertir una matriz utilizando algún método como lo es la Eliminación de Gauss-Jordan. Esta operación en particular es de orden de $\mathcal{O}(n^3)$ flops, lo que puede ser considerablemente

superior al orden del Método del Gradiente, sobre todo cuando la dimensión del problema (valor de n) es grande. Es por esto que si bien Newton tiene un orden de la cantidad de iteraciones inferior al del Gradiente, los tiempos invertidos en cada iteración puede ser miles de veces superior.

Con todo esto, el tiempo total de cada uno de los métodos tiene un tiempo total que es proporcional a:

$$\text{Tiempo total Gradiente} = \text{cte}_G \times \log\left(\frac{1}{\epsilon}\right) \times n$$

$$\text{Tiempo total Newton} = \text{cte}_N \times \log\left(\log\left(\frac{1}{\epsilon}\right)\right) \times n^3$$

donde cte_G y cte_N son constantes que dependen de cada método pero no de la dimensión n .

Esto muestra el balance que se produce entre número de iteraciones y costo por iteración. Para n grande, el factor n^3 puede no ser suficientemente compensado por el doble logaritmo y puede resultar que el Método del Gradiente termine usando menos tiempo efectivo.

Cápsula de video 2.4.2: Costo computacional de los métodos

Profesor Jorge Vera explica en clases las diferencias en costo computacional de los métodos de gradiente y Newton:

<https://youtu.be/olweRbMOXxw>

2.5. Linesearch: Selección del paso

Los algoritmos de descenso vistos requieren resolver la etapa del “paso de avance” o *linesearch*, que consiste en buscar el mínimo en un espacio unidireccional (una línea). Esta etapa es en la que se busca el valor de λ^k que resuelva:

$$\min_{\lambda \geq 0} f(x^k + \lambda d^k)$$

Meter un subproblema de optimización dentro de cada una de las iteraciones del problema principal puede ser inconveniente en término del tiempo en que se ejecuta el algoritmo, esto a pesar que es un problema de optimización en una sola variable. Ante esto, cabe decir que existe la posibilidad de relajar la búsqueda de este paso λ^k .

Se han desarrollado varios métodos para realizar el *linesearch*, que trabajan sobre la función $h(\lambda) = f(x^k + \lambda d^k)$.²

2.5.1. Métodos de búsqueda

Estos métodos utilizan procesos de aproximación rápida para obtener una solución al paso. Revisemos brevemente algunos de ellos.

Búsqueda de la Sección Áurea

Este método utiliza un oráculo de orden cero (solo usa solo valores de f) y su desempeño es eficaz en la medida que f sea “unimodal”, vale decir, que tiene solo un mínimo local en el intervalo de búsqueda $[x_1, x_3]$. La búsqueda se realiza del siguiente modo (ver Figura 2.4): En primer lugar, se comienza con los puntos x_1 y x_3 que definen al intervalo, posteriormente se elige un punto x_2 , ente x_1 y x_3 tal que esté a unidades a la derecha de x_1 y b unidades a la izquierda de x_3 . También se elige un punto x_4 que esté c unidades a la derecha de x_2 .

Estos puntos deben estar esparcidos por la línea de búsqueda, tal que se eligen del siguiente modo que:

²En *Python*, la subrutina del *linesearch* se puede hacer mediante la función de la librería *Spicy*: `scipy.optimize.fminbound`. Y con *Matlab*: `fminbnd`. Ambas utilizan condiciones de tipo Wolfe-Armijo.

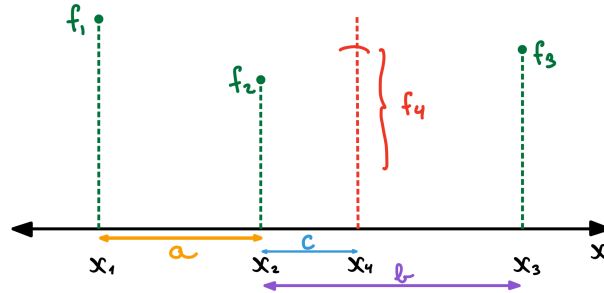


Figura 2.4: Búsqueda de Sección Áurea

$$\frac{c}{a} = \frac{a}{b} \quad \text{y que} \quad \frac{c}{b-c} = \frac{a}{b}$$

De esto se obtiene que:

$$\left(\frac{b}{a}\right)^2 - \left(\frac{b}{a}\right) = 1$$

Resolviendo esta ecuación cuadrática, se obtiene que la razón entre b y a debe ser el número áureo:

$$\frac{b}{a} = \frac{1 + \sqrt{5}}{2} = 1,618 \quad \longrightarrow \quad \frac{a}{c} = \frac{b}{a} = \phi$$

Para elegir los siguientes puntos a evaluar se utiliza un algoritmo que estudie siempre tres puntos y busque un cuarto punto entre medio según la sección áurea. En el caso que se muestra en la imagen, se debe elegir el conjunto de los próximos tres puntos sobre los que hacer la búsqueda de sección áurea del siguiente modo:

1. Si $f(x_4) < f(x_2) \longrightarrow$ seguir buscando sobre el conjunto $\{x_2, x_4, x_3\}$
2. Si $f(x_4) > f(x_2) \longrightarrow$ seguir buscando sobre el conjunto $\{x_1, x_2, x_4\}$

Bisección

Este método utiliza un oráculo de primer orden (usa el valor de las derivadas de f) y su desempeño es eficaz en la medida que f sea “unimodal”, vale decir, que tiene solo un mínimo local en el intervalo de búsqueda $[t_1, t_3]$, de este modo, se busca el punto t^* , tal que $h'(t^*) \approx 0$. La búsqueda se realiza del siguiente modo (ver Figura 2.5): En primer lugar, se comienza evaluando la derivada en los puntos t_1 y t_3 que definen al intervalo, posteriormente se evalúa la derivada en un punto t_2 intermedio y equidistante de t_1 y t_3 . En caso que $h'(t_2) > 0$, entonces se continúa iterando del mismo modo, pero esta vez buscando entre t_1 y t_2 . Si $h'(t_2) < 0$ entonces se continúa iterando del mismo modo, buscando entre t_2 y t_3 .

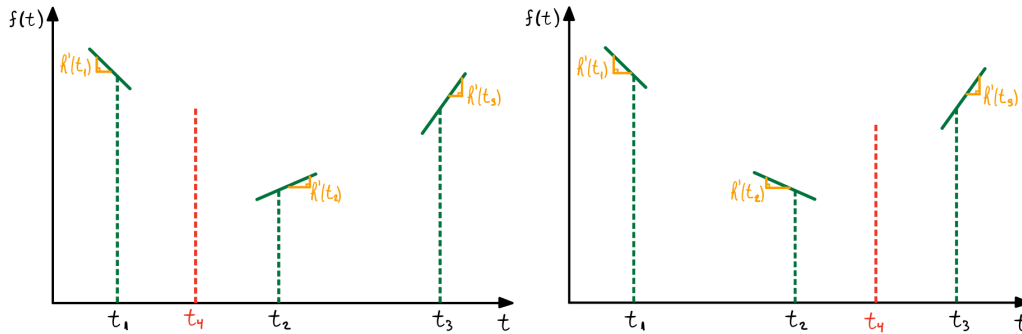


Figura 2.5: Bisección

Interpolación

Supongamos tenemos dos puntos, t_1, t_2 tales que conocemos $h(t_1), h(t_2)$ y $h'(t_1)$. Se puede interpolar una función cuadrática $g(t)$ y minimizarla.

Sea $g(t) = at^2 + bt + c$. Se puede formar el siguiente sistema de ecuaciones:

$$\begin{aligned} h(t_1) &= a(t_1)^2 + b(t_1) + c \\ h(t_2) &= a(t_2)^2 + b(t_2) + c \\ h'(t_1) &= 2a(t_1) + b \end{aligned}$$

Resolviendo este sistema tenemos a, b, c y así conocemos g . Ahora minimizamos $g(t) = at^2 + bt + c$. Si f es unimodal o convexa, entonces $a > 0$ y el mínimo se obtiene derivando $g(t)$ e igualando a 0: $g'(t) = 2at + b = 0$, de donde se obtiene que el mínimo t_3 de esta interpolación cuadrática es:

$$t_3 = -\frac{b}{2a}$$

Se evalúa $h'(t_3)$ y se decide continuar con el intervalo $[t_1, t_3]$ o $[t_3, t_2]$ y así sucesivamente. Se podrían hacer cosas más complejas como interpolación cúbica, pero que hace que el algoritmo sea más lento por iteración.

2.5.2. Condición de Wolfe-Armijo

Los métodos anteriores pueden encontrar una solución al paso con toda la precisión que se desee. Ahora haremos notar que, en realidad, no es necesario tener una solución precisa, basta que se cumplan ciertas condiciones. Esto fue establecido por P. Wolfe y R. Armijo en la década de 1970 y lo revisaremos brevemente ahora.

Sea la función $h(\lambda) = f(x^k + \lambda d^k)$ y $h'(\lambda) = \nabla f(x^k + \lambda d^k)^T d^k$. Un valor λ cumple con las condiciones de Wolfe y Armijo si:

$$\begin{aligned} h(\lambda) &\leq h(0) + c_1 \lambda h'(0) && \text{(condición de Armijo)} \\ h'(\lambda) &\geq c_2 h'(0) && \text{(condición de Wolfe)} \end{aligned}$$

Donde $0 < c_1 < c_2 < 1$. La primera condición corresponde a que $h(\lambda)$ está acotado por abajo por la línea $h(0) + c_1 \lambda h'(0)$, mostrada en azul en la Figura 2.6. La segunda condición dice que la pendiente en el nuevo punto debe ser menos negativa que la del punto original en la dirección de la línea en cuestión (pendiente mostrada en verde). Todo esto da un intervalo donde se cumplen ambas condiciones y se puede mostrar que utilizando este paso, tanto el método de gradiente como el método de Newton convergen.

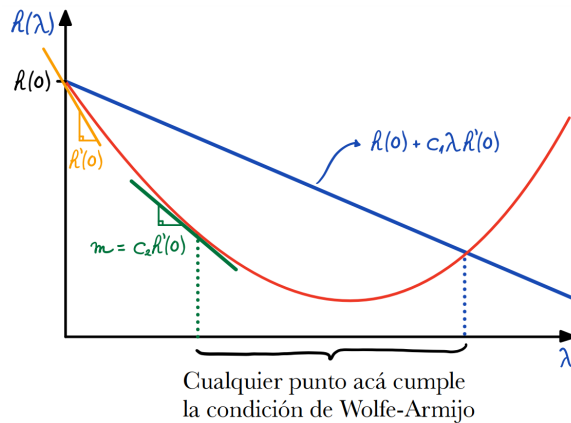


Figura 2.6: Condición de Wolf-Armijo

La condición de Armijo garantiza un mínimo decrecimiento de la función objetivo y la condición de Wolfe garantiza un mínimo avance. Los valores adecuados de c_1 y c_2 se pueden encontrar en la literatura. Suelen elegirse en base a la experiencia según el problema. El valor de λ se obtiene iterando con *backtracking*, de

modo que se parte con un λ inicial y se va iterando este valor hasta que se encuentra uno que satisfaga las condiciones. Esto típicamente se puede hacer como:

$$\lambda_{k+1} = \rho \lambda_k, \quad 0 < \rho < 1$$

Si es que cae al lado izquierdo del intervalo, ahí se utiliza un valor de $\rho > 1$ para acercarse al intervalo.

Ejemplo algoritmo de Wolfe-Armijo con backtraking

0. Data: Sea $\lambda^0 = b$, $k = 0$
1. if $h(\lambda^k) \leq h(0) + \lambda^k c_1 h'(\lambda^k)$, entonces se cumple Armijo **do**
 if $h'(\lambda^k) \geq c_2 h'(0)$, entonces se cumple Wolfe **do**
 STOP.
 $\lambda^{k+1} \leftarrow (\lambda^k + \lambda^{k+1})/2$
 Go to 1.
2. $\lambda_{k+1} = \rho \lambda_k$, $0 < \rho < 1$, $k \leftarrow k + 1$, Go to 1.

Cabe mencionar que el ejemplo de la figura es de una función convexa y en esa situación existirá siempre un intervalo en donde se puede elegir el paso. Sin embargo, si la función no es convexa, puede haber más de un intervalo válido.

El punto que se obtiene mediante Wolfe-Armijo no es un punto en que necesariamente $h'(\lambda) \approx 0$, tal como los otros métodos mencionados. Es por esto que no es necesario optimizar y se puede llegar a un punto más relajado, pero lo importante es que es un punto que tiene un valor inferior al punto inicial y que es económico de obtener en términos computacionales.

Cápsula de video 2.5.1: Linesearch y Condiciones de Wolfe-Armijo

Profesor Jorge Vera explica en clases las Condiciones de Wolfe y Armijo y la importancia del problema de Linesearch:
<https://youtu.be/apoI2S0uvTc>

2.6. Métodos Quasi-Newton

Como comentamos en la sección anterior, el Método de Newton es muy potente debido a su convergencia cuadrática. Sin embargo, el costo por iteración es muy alto y esto puede traducirse en ineficiencias. Desde el comienzo de los desarrollos de algoritmos computacionales surgió la meta de desarrollar métodos que tuvieran propiedades similares al método de Newton pero con un menor costo por iteración. Esto dió origen a los métodos Quasi-Newton que reviseramos, brevemente, ahora.

2.6.1. Métodos Quasi-Newton: BFGS

Un Método Quasi-Newton busca disminuir el costo computacional de invertir la matriz en cada iteración, a costa de hacer más iteraciones. Para esto, se **aproxima el hessiano** $\nabla^2 f(x^k)$ a una matriz “similar” B^k , igualmente simétrica y definida positiva, pero con mejores propiedades para invertirla. De hecho, la inversa se podrá calcular con una complejidad orden $\mathcal{O}(n^2)$. De este modo, la dirección de descenso queda definida como:

$$d^k = -[B^k]^{-1} \nabla f(x^k)$$

Y el paso siguiente como:

$$x^{k+1} = x^k - \lambda [B^k]^{-1} \nabla f(x^k)$$

Para definir B^k adecuadamente se recurre al Teorema del Valor Medio, como explicamos a continuación. Utilizando los gradientes y el hessiano de f , entre dos soluciones consecutivas de las iteraciones x^k y x^{k+1} podemos plantear lo siguiente:

$$\nabla f(x^{k+1}) - \nabla f(x^k) \approx \nabla^2 f(x^{k+1})(x^{k+1} - x^k)$$

Dado que la matriz $\nabla^2 f(x^{k+1})$ cumple esa propiedad y queremos que B^k tenga las mismas propiedades del Hessiano, imponemos elegir B^k tal que cumpla con:

$$\nabla f(x^{k+1}) - \nabla f(x^k) = B^{k+1}(x^{k+1} - x^k)$$

Sean: $s_k = x^{k+1} - x^k$ y $y_k = \nabla f(x^{k+1}) - \nabla f(x^k)$. Luego, la expresión anteriormente encontrada se puede escribir como:

$$y_k = B^{k+1}s_k$$

Esta expresión se conoce como “la condición de secante”. Además se requiere que B^{k+1} sea definida positiva y simétrica.

Como la condición de secante tiene infinitas soluciones, se calcula el B^{k+1} de modo que sea lo más parecido posible a B^k , por lo que se resuelve el siguiente problema de optimización:

$$\begin{aligned} \min_B \quad & \|B - B^k\| \\ \text{s.a.} \quad & B = B^T \\ & Bs_k = y_k \end{aligned}$$

Obtener una solución de este problema que sea definida positiva solo es posible si se cumple la condición de curvatura: $s_k^T y_k > 0$. Al final de esta sección (★) se discute esto en más detalle.

La resolución de este problema de optimización requiere conceptos avanzados de análisis matricial (se resuelve con Lagrange y está en el libro guía de J. Nocedal), pero lo importante es que tiene una solución explícita si se utiliza una norma matricial adecuada. Con un desarrollo idéntico al expuesto, se puede formular una aproximación para el inverso del hessiano $[\nabla^2 f(x^k)]^{-1}$, ya que la inversa de B^k , que es $(B^k)^{-1}$, es lo que realmente interesa. De este modo, se define H^k como $(B^k)^{-1}$, donde el valor de H^{k+1} está dado por la siguiente fórmula:

$$H^{k+1} = (I - \rho_k s_k y_k^T) H^k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T, \quad \text{donde} \quad \rho_k = \frac{1}{y_k^T s_k} > 0$$

Esto se conoce como la fórmula de Broyden-Fletcher-Goldfarb-Shanno o BFGS, por sus creadores, que la desarrollaron en la década de 1980. Como $\rho_k > 0$, entonces la matriz a calcular H^{k+1} es definida positiva.

Algoritmo BFGS

0. Sea $x^0 \in \mathbb{R}^n, H^0 \in \mathbb{R}^{n \times n}, k = 0, \epsilon > 0$.
1. Dirección de descenso: $d^k = -H^k \nabla f(x^k)$
2. Si $\|\nabla f(x^k)\| \leq \epsilon$: STOP.
3. Calcular λ por *linesearch* que cumpla Wolfe-Armijo.
4. Definir $x^{k+1} = x^k + \lambda^k d^k$.
5. Calcular H^{k+1} por BFGS, $k \leftarrow k + 1$. Go to 1.

Para el valor de H^0 , se puede calcular el hessiano en la primera iteración $[\nabla^2 f(x^0)]^{-1}$ u otra aproximación, incluso puede ser $H^0 = I$. Esto último puede pensarse como muy ineficiente pero funciona igual ya que el método poco a poco va añadiendo información de segundo orden cuando usa la fórmula basada en el Teorema del Valor Medio.

2.6.2. Eficiencia de BFGS

Por otro lado, así como el Método del Gradiente evaluaba $\mathcal{O}(n)$ “flops” en cada iteración, y Newton $\mathcal{O}(n^3)$, BFGS evalúa $\mathcal{O}(n^2)$ “flops”, lo que mostramos a continuación:

1. $d^k = -H^k \nabla f(x^k)$, es $\mathcal{O}(n^2)$
2. $\rho_k = y_k^T s_k$, es $\mathcal{O}(n)$
3. $\rho_k = s_k s_k^T$ es $\mathcal{O}(n^2)$
4. Cuidando de nunca multiplicar una matriz por otra matriz, ya que eso lo aumenta $\mathcal{O}(n^3)$: cuando se calcula $H^k(I - \rho_k y_k s_k^T)$, se debe hacer en el siguiente orden inteligente: $(I - \rho_k s_k y_k^T)(H^k - \rho_k(H^k y_k) s_k^T)$, de modo que:
 - i. $w = H^k y_k$, es $\mathcal{O}(n^2)$
 - ii. $V = w s_k^T$, es $\mathcal{O}(n^2)$
 - iii. $U = H^k - \rho V$, es $\mathcal{O}(n^2)$
 - iv. $z^T = y_k^T U$, es $\mathcal{O}(n^2)$
 - v. $T = s_k z^T$, es $\mathcal{O}(n^2)$
 - vi. $H^{k+1} = U - \rho T$, es $\mathcal{O}(n^2)$

Cápsula de video 2.6.1: Costo Computacional de BFGS

Profesor Jorge Vera explica en clases por qué el costo computacional por iteración del Método BFGS es $\mathcal{O}(n^2)$.

<https://youtu.be/h3JvfViUxUg>

Se puede probar que BFGS converge a un mínimo local, si el punto inicial está “suficientemente cerca” del óptimo, y la función es “bien comportada” y el hessiano es definido positivo, localmente en torno al óptimo, y la función es convexa. La velocidad de convergencia no va a ser mejor que el método de Newton, pero si que la del método del Gradiente. Si x^* es mínimo local, $\nabla^2 f(x^*)$ es definida positiva, y existe $r > 0$ tal que:

$$\|\nabla^2 f(x) - \nabla^2 f(y)\| \leq L\|x - y\|, \quad \forall x, y \in B(x^*, r)$$

entonces se puede mostrar que si $\{x^k\}_k$ es la sucesión generada por BFGS, se cumple:

$$\lim_{k \rightarrow \infty} \frac{\|x^{k+1} - x^*\|}{\|x^k - x^*\|} = 0$$

Esta relación implica que el error de la iteración $k + 1$, e^{k+1} , se va más rápido a cero que el error de la iteración k , e^k . De hecho, existe una sucesión de valores $\{\alpha_k\}_k$, con $\alpha_k \rightarrow 0$, cuando $k \rightarrow \infty$, tal que:

$$\|x^{k+1} - x^*\| \leq \alpha_k \|x^k - x^*\|, \quad k = 0, 1, \dots$$

Notar que esto es similar a la convergencia lineal, pero con la diferencia que el factor que multiplica el lado derecho ya no es una constante, sino que una sucesión que converge a cero, por lo que el error disminuye más rápido que a una tasa fija, pero no más rápido que el cuadrático de Newton. Esto se llama convergencia súper-lineal, lo que es mucho mejor que convergencia lineal pero no es tan rápido como la convergencia cuadrática. En general sucede que BFGS realiza más iteraciones que Newton pero menos que el Gradiente. Al final, igual es mucho mejor que Newton para problemas grandes debido a que es $\mathcal{O}(n^2)$, que es mucho mejor que Newton ($\mathcal{O}(n^3)$). En general es tan bueno que en *Matlab* y en *Python* se suele utilizar BFGS por defecto para problemas no restringidos. No se puede saber si BFGS es de orden $\mathcal{O}(\frac{1}{\epsilon})$.

Uno de los problemas importantes es que para n grande hay que almacenar toda la matriz H^k , para eso se utiliza el limited memory BFGS (L-BFGS) que guarda en memoria solo una parte de H^k , con esto no se acumulan todos los s_k, y_k , sino solo una cantidad fija de ellos, lo que mantiene la aproximación de H^k . Esta “aproximación de la aproximación” suele funcionar muy bien.

Apéndice

Demostración de que BFGS con la regla de Wolfe-Armijo cumple con $y_k^T s_k > 0$:

En primer lugar, hay que considerar los siguientes tres aspectos:

$$(1): s^k = x^{k+1} - x^k$$

$$(2): x^{k+1} = x^k + \lambda_k d^k$$

$$(1) + (2): d^k = \frac{s^k}{\lambda_k} \Leftrightarrow d^k \lambda_k = s^k$$

$$(3): y_k = \nabla f(x^{k+1}) - \nabla f(x^k)$$

Donde d^k es dirección de descenso en x^k . La idea es partir de la regla de Wolfe-Armijo, y demostrar que $y_k^T s_k > 0$, para lo cual hay que llegar a una expresión similar a (3) multiplicada por s_k : Por la regla de Wolfe-Armijo, el paso λ_k satisface:

$$\begin{aligned} \nabla f(x^k + \lambda_k d^k)^T d^k &\geq c_2 \nabla f(x^k)^T d^k \\ \nabla f(x^{k+1})^T d^k &\geq c_2 \nabla f(x^k)^T d^k \\ \nabla f(x^{k+1})^T \frac{s^k}{\lambda_k} &\geq c_2 \nabla f(x^k)^T d^k \\ \nabla f(x^{k+1})^T s^k &\geq c_2 \lambda_k \nabla f(x^k)^T d^k && / - \nabla f(x^k)^T s^k \\ \nabla f(x^{k+1})^T s^k - \nabla f(x^k)^T s^k &\geq c_2 \lambda_k \nabla f(x^k)^T d^k - \nabla f(x^k)^T s^k \\ (\nabla f(x^{k+1})^T - \nabla f(x^k)^T) s^k &\geq c_2 \lambda_k \nabla f(x^k)^T d^k - \nabla f(x^k)^T d^k \lambda_k \\ y_k^T s^k &\geq (c_2 - 1) \lambda_k \nabla f(x^k)^T d^k \end{aligned}$$

Por último, como d^k es dirección de descenso en x^k , tenemos que $\nabla f(x^k)^T d^k < 0$, y como $(c_2 - 1) < 0$, es posible concluir que el lado derecho es positivo.

Todo lo expuesto en este capítulo corresponde a algunos de los métodos “clásicos”, desarrollados mucho tiempo atrás, aún así, la idea de avanzar por una dirección de descenso, usar gradiente y hessiano e iterar está muy vigente. Pero la optimización moderna requiere enfoques nuevos para los nuevos problemas del Internet, el *Big Data*, etc. Estos problemas se han reenfocado en los últimos tiempos. Ese es el próximo capítulo.

2.7. Ejemplos

2.7.1. Aplicación en IA: Optimización de Redes Neuronales

Las Redes Neuronales son un modelo de gran importancia en *Machine Learning* y son usadas para abordar problemas de predicción, reconocimiento de patrones, etc. La idea fundamental de una red neuronal está inspirada en la estructura del cerebro: una red de neuronas interconectadas que se envían información. Se recomienda para entender en más detalle, ver en el canal de Youtube 3Blue1Brown. De ese modo, se implementa un mecanismo de cálculo que permite estimar una función matemática f , de n variables, de la que sólo se conoce su valor en algunos puntos. Presentamos aquí un modelo básico de una red neuronal: la red neuronal consiste en un conjunto de nodos de “input”, $I = \{1, 2, \dots, n\}$, un conjunto de nodos intermedios, $C = \{1, \dots, m\}$, y un nodo final, s , que entrega el resultado. Los nodos I están conectados con todos los nodos C , y los nodos C están conectados con el nodo s .

¿Cómo “calcula” la red? En forma general, cada nodo (neurona) de la red recibe un input numérico t , y genera un output, también numérico, $g(t)$. La función g es una función simple de evaluar y en este caso asumiremos que una “sigmoides”, más específicamente, la siguiente:

$$g(t) = \tanh(t) \quad (\text{función tangente hiperbólica})$$

En forma más específica (Figura 2.7), cada neurona de “input” $i \in I$ recibe un valor x_i (las coordenadas del punto donde vamos a evaluar), y produce el output $y_i = g(x_i)$. Después, cada neurona de la capa intermedia, $k \in C$, recibe un input igual a $\sum_{i=1}^n w_{ik} y_i$, donde los números w_{ik} son pesos asociados a las conexiones. Con ese input, la neurona $k \in C$ genera un output igual a

$$z_k = g\left(\sum_{i=1}^n w_{ik} y_i\right), k \in C$$

Luego, esos valores z_k , $k \in C$, son alimentados a la neurona final s , la que recibe un input igual a $\sum_{k=1}^m p_{ks} z_k$, donde los p_{ks} son pesos asociados a las conexiones de los nodos de C hacia s , y produce el output final igual a

$$v = \sum_{k=1}^m p_{ks} z_k$$

(para simplificar, asumiremos que la última neurona no aplica la función g). Resumiendo entonces: la red completa recibe como input valores $(x_1, \dots, x_n)$ y produce v . Obviamente el cálculo depende de los pesos w_{ik} y p_{ks} , distintos pesos darán distintos outputs. La Figura 2.7 muestra el esquema de la red que hemos descrito.

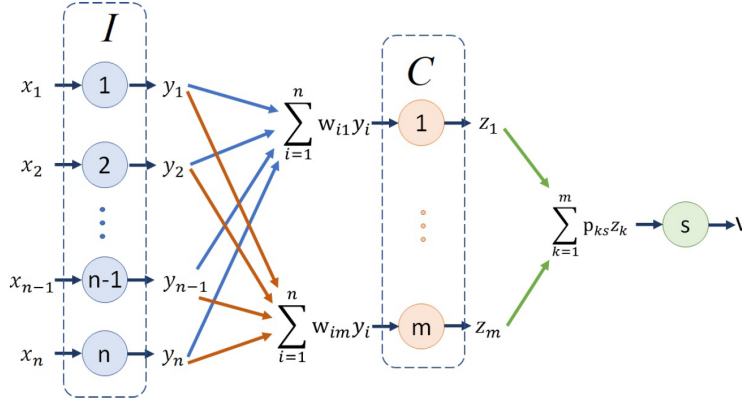


Figura 2.7: Esquema de Red Neuronal (dibujo de D. Cariaga)

Ahora queremos “entrenar” esa red neuronal para que “aprenda” la función f . Para esto, supongamos que nos han entregado observaciones de la función f para r distintos puntos. Es decir, conocemos valores $\bar{x}^l = (\bar{x}_1^l, \dots, \bar{x}_n^l)$, $l = 1, \dots, r$ y $\bar{v}^l = f(\bar{x}^l) = f(\bar{x}_1^l, \dots, \bar{x}_n^l)$.

Entonces, se entregan los vectores $\bar{x}^1, \dots, \bar{x}^r$ como input a la red y esta calcula outputs $v^1, \dots, v^r$, que no tiene por qué ser iguales a $\bar{v}^1, \dots, \bar{v}^r$ ya que lo que calcula la red depende de los valores que se hayan dado a los pesos w_{ik} y p_{ks} . Entonces, el problema a resolver consiste en determinar los mejores valores para estos pesos w_{ik} y p_{ks} de modo tal que los valores v^l , calculados por la red, sean lo más parecidos posibles a los valores $\bar{v}^l$, conocidos como datos. De este modo se “entrena” la red la que, después, debiera ser capaz de estimar la función f para otros inputs diferentes.

Este problema de entrenamiento puede ser formulado mediante un modelo de optimización de mínimos cuadrados, es decir, que minimiza el error cuadrático entre lo que calcula la red y el verdadero valor. El modelo es el siguiente:

$$\begin{aligned} \min_{w,p} \quad & \sum_{l=1}^r (v^l - \bar{v}^l)^2 \\ \text{s.a.} \quad & v^l = \sum_{k=1}^m p_{ks} z_k^l, \quad l = 1, \dots, r \\ & z_k^l = g \left(\sum_{i=1}^n w_i^k y_i^l \right), \quad l = 1, \dots, r, \quad k = 1, \dots, m \\ & y_i^l = g(\bar{x}_i^l), \quad i = 1, \dots, n, \quad l = 1, \dots, r \end{aligned}$$

Este problema de optimización con restricciones se puede reescribir el problema anterior como un problema irrestricto de optimización. Para ello considere las siguientes sustituciones para reescribir el valor de v^l para algún $l \in \{1, \dots, r\}$ en particular:

$$v^l = \sum_{k=1}^m p_{ks} z_k^l = \sum_{k=1}^m p_{ks} g \left(\sum_{i=1}^n w_i^k y_i^l \right) = \sum_{k=1}^m p_{ks} g \left(\sum_{i=1}^n w_i^k g(\bar{x}_i^l) \right)$$

Reemplazando este valor de v^l en la función objetivo se obtiene el problema planteado en su forma sin restricciones:

$$\min_{w,p} \sum_{l=1}^r \left(\sum_{k=1}^m p_{ks} g \left(\sum_{i=1}^n w_i^k g(\tilde{x}_i^l) \right) - \bar{v}^l \right)^2$$

Este es un problema de optimización que se podría abordar mediante el Método del Gradiente en *Python*. Para esto, en primer lugar, se necesita definir la función. Para esto, debe tenerse presente que el vector $\tilde{x}_i^l$ es constante, pues es el input que recibe la red neuronal. Las variables en este problema son los parámetros p_{ks} y w_i^k que se desean calibrar. Con esto en mente, se define el vector y que contiene todas las variables. Esto es: $y = [p, w] = [p_1, p_2, \dots, p_n, w_1, w_2, \dots, w_{n^2}]^3$ Luego, la función se puede definir como: **1.** $f(y, B)$. Las tres sumatorias de la función se puede formular en formas de “loop” que van sumando sucesivamente. El primer “loop” corresponde a la sumatoria con índice l que va entre 1 y r , la segunda a la de índice k entre 1 y n y la tercera a la de índice i , también entre 1 y n^4 :

```
def f(y, B):
    r, n_cols = B.shape
    n = n_cols - 1
    p, w = y[0:n], y[n:n**2 + n]

    suma_l = 0
    for l in range(r): # 1ra sumatoria
        suma_k = 0
        for k in range(n): # 2da sumatoria
            suma_i = 0
            for i in range(n): # 3ra sumatoria
                suma_i += w[n*k+i] * tanh(B[l, i]) # Suma de la 3ra sumatoria
            suma_k += p[k] * tanh(suma_i) # Suma de la 2da sumatoria
        suma_l += (suma_k - B[l, n]) ** 2 # Suma de la 1ra sumatoria
    return suma_l
```

Para aplicar el método del gradiente, se puede tener en cuenta que, como la función se puede evaluar la función en cualquier punto y , entonces es posible obtener el gradiente utilizando “derivación numérica”⁵. Para esto, basta recordar que la definición de la derivada de una función respecto a cualquier variable. Para este problema, esto es de la siguiente forma:

$$\frac{\partial f(y, B)}{\partial y_i} = \lim_{\Delta \rightarrow 0} \frac{f([y_1, \dots, y_i + \Delta, \dots, y_{n^2+n}], B) - f(y, B)}{\Delta}, \quad \forall i \in \{1, \dots, n^2 + n\}$$

Esto mismo se puede implementar en *Python*. En la función `nabla(y, B)` se calcula el gradiente de $f(y, B)$ respecto a $y = [p, w]$. Para esto, en primer lugar se guarda el valor original de la función $f(y, B)$ en `fval_0` y para cada una de las entradas de y se calculó la derivada direccional.

```
def nabla(y, B):
    r, n_cols = B.shape
    n = n_cols - 1

    grad, delta, fval_0 = np.zeros(n+n**2), 0.001, f(y, B)

    for i in range(n + n**2):
        y[i] = y[i] + delta
        grad[i] = (f(y, B) - fval_0) / delta
        y[i] = y[i] - delta

    return grad
```

³Acá la notación puede ser confusa. En realidad, $[p, w]$ corresponde a un vector y no a una fila. Formalmente, se debería escribir como $y = [p^T, w^T]^T$.

⁴Notar que en *Python* `range(n)` genera números entre 0 y $n - 1$, que para efectos prácticos es equivalente a que sea entre 1 y n .

⁵Una forma más eficiente es propuesta en el Anexo 2

Con estas funciones definidas, es posible utilizar un algoritmo análogo al mostrado para el Gradiente, para así calibrar los parámetros p y w que se encuentran en el vector y . Esto se puede hacer mediante el siguiente código:

```
import numpy as np
from numpy import tanh
import scipy.optimize
from apendice import datos_red_neuronal

def entrenar_red_neuronal(B, epsilon, n_i, y_i):
    """
    Funcion que calibra los parametros de una red neuronal utilizando
    el Metodo del Gradiente.
    """
    def funcion2T(lambda_, y, h, B): return f(y+lambda_*h, B)

    # Se definen los parametros iniciales para comenzar el algoritmo
    y, k, stop = y_i, 0, False
    grad = nabla(y, B)

    while (not stop) and (k < n_i):
        # Direccion de descenso
        h = - grad
        # Linesearch
        lambda_ = scipy.optimize.fminbound(funcion2T, 0, 10, args=(y, h, B))
        # Actualizacion de y, del valor de la funcion objetivo y del gradiente
        y = y + lambda_*h
        val, grad = f(y, B), nabla(y, B)
        print('valor:', val)
        # Criterio de parada
        if np.linalg.norm(grad, 2) < epsilon: stop = True
        k += 1

    return val, y

if __name__ == '__main__':
    n = 10
    B1, B2 = datos_red_neuronal(29132, n)
    y_i = np.random.rand(n + n ** 2)

    suma_i1, suma_i2 = f(y_i, B1), f(y_i, B2)
    print('suma_inicial_B1:', suma_i1, '|suma_inicial_B2:', suma_i2)

    val, y = entrenar_red_neuronal(B1, 0.00001, 4, y_i)

    suma_f1, suma_f2 = f(y, B1), f(y, B2)
    print('suma_final_B1:', suma_f1, '|suma_final_B2:', suma_f2)
```

Se podría hacer algo análogo con el Método de Newton, calculando numéricamente el hessiano. Sin embargo, para este tipo de problemas, Newton no resulta conveniente. Esto se debe a que al estar la función de tangente hiperbólico, que no es convexa, entonces Newton tendrá problemas. Este mismo problema es causante que puedan existir muchos óptimos locales, por lo que el punto de partida es determinante para encontrar una buena solución.

2.7.2. Aplicación: Optimización en doblamiento de proteínas

En biología se ha estudiado desde hace tiempo la forma en que las proteínas se combinan para dar origen a diversos tejidos en un organismo, entre otras cosas. La forma espacial que adquiere una proteína tiene que ver con la constitución de sus aminoácidos, los cuales se atraen o repelen según sus características. Esto puede ser modelado como un problema de optimización y abordado con los métodos que hemos estudiado,

pero es un problema desafiante.

Para esto, vamos a considerar un modelo conceptual de la proteína, el que está formado por una secuencia de elementos rígidos. Dichos elementos están unidos por sus extremos en donde se pueden articular. Los distintos elementos se atraen o repelen según la acción de fuerzas eléctricas y nucleares, de acuerdo a características conocidas. Para simplificar, asumiremos que las fuerzas se aplican en los puntos centrales de los elementos. La figura 2.8 muestra un modelo simplificado de una proteína en 2 dimensiones.

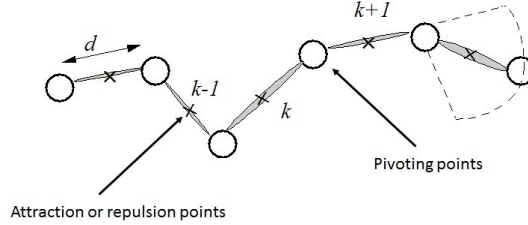


Figura 2.8: Modelo de una Proteína.

Asumamos que tenemos K elementos, y que los puntos extremos del elemento k son (x_{k-1}, y_{k-1}) y (x_k, y_k) . Asumimos igualmente, también para simplificar, que todos los elementos tienen un largo fijo igual a d . Luego, una restricción del sistema es que:

$$(x_k - x_{k-1})^2 + (y_k - y_{k-1})^2 = d^2, \quad k = 1, \dots, K.$$

También especificaremos que el punto inicial, (x_0, y_0) es fijo y conocido.

Para modelar las fuerzas eléctricas y nucleares que producen el doblamiento de la proteína, asumimos que todo el sistema trata de alcanzar un estado de mínima energía. Esta energía está dada por:

$$\sum_{i \neq j} \frac{a_{ij}}{dist_{ij}}$$

donde $dist_{ij}$ es la distancia entre el elemento i y el elemento j , y a_{ij} es un coeficiente conocido que depende que las características del elemento. La distancia la calculamos entre los puntos centrales de los segmentos. Lo que se asume es que los elementos de la proteína se ubican espacialmente de modo de minimizar la energía.

Para modelar el problema, vamos a representar las atracciones y repulsiones en los puntos centrales, como se indicó. Para esto, usaremos variables (u, v) para representar estos puntos centrales y la distancia la calculamos entre estos los puntos centrales de los segmentos. Sea:

$$\begin{pmatrix} u_k \\ v_k \end{pmatrix} = \frac{1}{2} \left[\begin{pmatrix} x_k \\ y_k \end{pmatrix} + \begin{pmatrix} x_{k-1} \\ y_{k-1} \end{pmatrix} \right].$$

Entonces,

$$dist_{ij} = \sqrt{(u_j - u_i)^2 + (v_j - v_i)^2},$$

Con esto, la función objetivo es:

$$\sum_{i \neq j} \frac{a_{ij}}{\sqrt{(u_j - u_i)^2 + (v_j - v_i)^2}}$$

El problema es, entonces, minimizar esa energía sujeto a las restricciones del largo de los elementos y la definición de los puntos centrales. En forma más específica:

$$\begin{aligned} \min \quad & \sum_{i \neq j} \frac{a_{ij}}{\sqrt{(u_j - u_i)^2 + (v_j - v_i)^2}} \\ \text{s.a.} \quad & \begin{pmatrix} u_k \\ v_k \end{pmatrix} = \frac{1}{2} \left[\begin{pmatrix} x_k \\ y_k \end{pmatrix} + \begin{pmatrix} x_{k-1} \\ y_{k-1} \end{pmatrix} \right] \\ & (x_k - x_{k-1})^2 + (y_k - y_{k-1})^2 = d^2, \quad k = 1, \dots, K. \end{aligned}$$

Este problema puede todavía simplificarse un poco ya que las variables (u, v) no son necesarias y pueden ser reemplazadas. Esto lleva al siguiente modelo:

$$\begin{aligned} \min \quad & \sum_{i \neq j} a_{ij} \left(\left(\frac{1}{2}(x_j - x_i) + \frac{1}{2}(x_{j+1} - x_{i+1}) \right)^2 + \left(\frac{1}{2}(y_j - y_i) + \frac{1}{2}(y_{j+1} - y_{i+1}) \right)^2 \right)^{-1/2} \\ \text{s.a.} \quad & (x_k - x_{k-1})^2 + (y_k - y_{k-1})^2 = d^2, \quad k = 1, \dots, K. \end{aligned}$$

Este es un problema de minimización de una función objetivo no lineal, pero sujeto a restricciones de igualdad. Vamos a modificar el modelo de modo tal que las restricciones de igualdad puedan ser eliminadas y den como resultado un modelo de optimización sin restricciones.

Hay varias formas de hacer eso, pero una es despejar x_k de las restricciones y reemplazar esa expresión en la función objetivo. Tenemos:

$$x_j = x_{j-1} + \sqrt{d^2 - (y_j - y_{j-1})^2}$$

Pero esto puede nuevamente aplicarse:

$$x_j = x_{j-2} + \sqrt{d^2 - (y_{j-1} - y_{j-2})^2} + \sqrt{d^2 - (y_j - y_{j-1})^2}$$

y así sucesivamente para llegar a:

$$x_j = x_0 + \sum_{k=0}^{j-1} \sqrt{d^2 - (y_{j-k} - y_{j-k-1})^2}$$

Esto nos lleva a:

$$\begin{aligned} \min_{x,y} \sum_{i \neq j} a_{ij} & \left(\left(\frac{1}{2} \left(\sum_{k=0}^{j-1} \sqrt{d^2 - (y_{j-k} - y_{j-k-1})^2} - \sum_{k=0}^{i-1} \sqrt{d^2 - (y_{i-k} - y_{i-k-1})^2} \right) + \right. \right. \\ & \left. \frac{1}{2} \left(\sum_{k=0}^j \sqrt{d^2 - (y_{j+1-k} - y_{j-k})^2} - \sum_{k=0}^i \sqrt{d^2 - (y_{i+1-k} - y_{i-k})^2} \right) \right)^2 + \\ & \left. \left(\frac{1}{2}(y_j - y_i) + \frac{1}{2}(y_{j+1} - y_{i+1}) \right)^2 \right)^{-1/2} \end{aligned}$$

que es, obviamente, una función muy no lineal, pero claramente diferenciable, excepto en combinaciones de puntos que puedan hacer nulo el denominador.

Ahora, discutimos las ventajas y desventajas que puedan tener los métodos que hemos estudiados, aplicados a este problema. Notemos que la función objetivo del problema puede ser compleja de derivar, a mano es casi imposible. En software de álgebra simbólica podría usarse, pero aún así posiblemente deban obtenerse los gradiente y Hessianos mediante rutinas de derivación numéricas. En todo caso, si hay K elementos en la molécula, que corresponden a aminoácidos, entonces el problema tiene K variables, las y . denotemos por f la función objetivo del problema. Entonces, ∇f será necesario si se usa el método del gradiente y es un vector de n componentes. Si queremos usar Newton, necesitaremos el Hessiano, una matriz de $n \times n$.

Por lo tanto, para poder comparar los métodos, sería útil tener una idea de cuánto puede llegar a ser n . Por ejemplo, en⁶ se describe el detalle biológico del problema y se indica que hay 20 aminoácidos fundamentales, que se combinan en cadenas desde 50 a 2000 elementos. En⁷ se investiga el número de aminoácidos en varias familias de proteínas y se dan número en el orden de varios cientos. Entonces, podemos asumir que n será un número de varios cientos, máximo en el orden de un par de miles. Esto es una cantidad de variables relativamente razonable para los métodos que hemos estudiado en el curso, en todo caso.

Mencionamos a continuación, algunas posibles ventajas y desventajas de los cuatro métodos frente a este problema:

⁶The Shape and Structure of Proteins, recuperado de <https://www.ncbi.nlm.nih.gov/books/NBK26830/> en 3 de septiembre de 2020.

⁷Axel Tiessen, Paulino Pérez-Rodríguez y Luis José Delaye-Arredondo, Mathematical modeling and comparison of protein size distribution in different plant, animal, fungal and microbial species reveals a negative correlation between protein size and protein number, thus providing insight into the evolution of proteomes, BMC Research Notes volume 5, (2012)

Método del gradiente:

Tiene la ventaja de ser el método que menos cálculos requiere. Hay que determinar el gradiente de la función y, dada la forma funcional que esta tiene, no es muy claro que se pueda hacer de forma fácil en forma analítica. La desventaja del método del gradiente es que es lento, en términos comparativos.

Método de Newton:

La gran ventaja de Newton es la convergencia cuadrática. Por otro lado, se requiere el Hessiano, adicionalmente al gradiente. Posiblemente es casi imposible encontrar una fórmula analítica para el Hessiano, así que habrá que recurrir a derivación numérica (tal como en el problemas de las preguntas 1, 2 y 3). Cabe notar que, según lo comentado arriba, el costo por iteración de Newton, que es $O(n^3)$, aún estará en el rango de lo manejable ya que n no será demasiado grande. Según el desarrollo del problema 4, n debe estar en el orden de decenas de miles para que comience a verse una diferencia apreciable en tiempo real de cómputo.

Método BFGS

Posiblemente es el mejor compromiso entre la lentitud del método del gradiente y la eficiencia de Newton. Las iteraciones serán relativamente rápidas y requieren sólo de los gradientes.

Un tema adicional, que afecta al método del gradiente, Newton y BFGS, es el problema del linesearch. Hacer un linesearch con muchas iteraciones puede ser complejo, dada la forma de la función. Un método como el de búsqueda de sección aurea puede ser más conveniente ya que requiere solo evaluaciones de f y no las derivadas.

2.7.3. Centro Analítico de un Poliedro

Considere el poliedro definido por el siguiente sistema de desigualdades lineales:

$$a_i^T x \leq b_i, i = 1, \dots, m$$

donde $a_i \in \mathbb{R}^n, i = 1, \dots, m$, donde A es la matriz cuyas filas son los vectores $a_i \in \mathbb{R}^n, i = 1, \dots, m$. Vamos a suponer que $P = \{x : Ax \leq b\}$ tiene interior no vacío y es un conjunto acotado. Entonces, como objeto geométrico tiene un “centro”. Una forma de encontrar uno de esos puntos especiales es calcular lo que se conoce como el “centro analítico” del poliedro. Para esto, consideramos la siguiente función, definida en el interior de P :

$$\Phi(x) = - \sum_{i=1}^m \log(b_i - a_i^T x)$$

Lo que nos interesa hacer es resolver el problema

$$\begin{array}{ll} \text{mín} & \Phi(x) \\ \text{s.a.} & x \in \text{int}(P) \end{array}$$

A primera vista, este parece un problema restringido pero en realidad no lo es. La función Φ sólo está definida dentro del poliedro, no es posible evaluarla fuera. Entonces, si pensamos en un algoritmo que parta en algún punto $x^0 \in \text{int}(P)$, entonces ningún punto genera debería estar fuera del dominio. Esto nos permite considerar el problema como si fuera no restringido.

El método favorito para abordar este problema es el método de Newton. Para su aplicación, necesitamos conocer $\nabla \Phi(x)$ y $\nabla^2 \Phi(x)$. Estos se pueden calcular derivando en forma ordenada. Lo primero a notar es que

$$\frac{\partial \Phi(x)}{\partial x_k} = \sum_{i=1}^m \frac{a_{ik}}{b_i - a_i^T x}$$

y esto es lo mismo que $A^T D(x)^{-1} e$, donde

$$D(x) = \begin{bmatrix} b_1 - a_1^T x & 0 & \cdots & 0 \\ 0 & b_2 - a_2^T x & \cdots & 0 \\ \vdots & \ddots & \ddots & 0 \\ \vdots & \ddots & \vdots & b_m - a_m^T x \end{bmatrix}$$

y e denota el vector de $\mathbb{R}^m$ cuyas componentes son todas iguales a 1. De aquí se obtiene que

$$\nabla\Phi(x) = A^T D(x)^{-1} e$$

Derivando una vez más se llega a que

$$\nabla^2\Phi(x) = A^T D(x)^{-2} A,$$

Ahora vamos a argumentar que Φ es estrictamente convexa en el interior de P si $m \geq n$ y el conjunto de vectores $a_1, \dots, a_m$ tiene rango completo. Esto se sigue del hecho que, como la definición de la función (por el logaritmo) garantiza que $b_i - a_i^T x$ sea siempre positivo, entonces $D(x)$ es siempre definida positiva, así como $D(x)^{-1}$ y $D(x)^{-2}$. Como A tiene rango máximo (sus columnas son linealmente independientes), entonces, para $u \neq 0$ se tiene que $u^T A^T D(x) A u = y^T D(x) y$, donde $y = Au$, y esto es positivo si $y \neq 0$. Pero como A es de rango máximo (sus columnas son linealmente independientes) y $u \neq 0$, entonces $y \neq 0$. Con Hessiano definido positivo, la función es estrictamente convexa.

Ahora, este es un pseudo-código para el Método de Newton aplicado al problema:

Algoritmo de Newton para el Centro Analítico:

0. Sea $x^0 \in \mathbb{R}^n$, $k = 0$, $\epsilon > 0$.
1. Sea $d^k = -[A^T D(x)^{-2} A]^{-1} A^T D(x)^{-1} e$
2. Si $\|A^T D(x)^{-1} e\| \leq \epsilon$ entonces STOP.
3. Sea λ^k solución óptima de

$$\min_{\lambda \geq 0} \Phi(x^k + \lambda d^k)$$

4. Sea $x^{k+1} = x^k + \lambda^k d^k$, $k \leftarrow k + 1$, Go to 1.

Como ya sabemos, el método requiere resolver el problema de linesearch. Por supuesto, podemos usar sólo una aproximación en la medida que cumpla con las condiciones de Wolfe-Armijo. Tenemos que, definiendo $h(\lambda) = \Phi(x^k + \lambda d^k) = -\sum_{i=1}^m \log(b_i - a_i^T(x^k + \lambda d^k))$ tenemos que

$$h'(\lambda) = \nabla\Phi(x^k + \lambda d^k)^T d^k = A^T D(x^k + \lambda d^k)^{-1} e$$

Luego, las condiciones de Wolfe-Armijo indican que se debe elegir λ de modo tal que

$$\begin{aligned} h(\lambda) &\leq h(0) + c_1 \lambda h'(0) \\ h'(\lambda) &\geq c_2 h'(0) \end{aligned}$$

que se traduce, para este caso particular, en:

$$\begin{aligned} \sum_{i=1}^m \log(b_i - a_i^T(x^k + \lambda d^k)) &\leq \sum_{i=1}^m \log(b_i - a_i^T x^k) + c_1 \lambda A^T D(x^k)^{-1} e \\ A^T D(x^k + \lambda d^k)^{-1} e &\geq c_2 A^T D(x^k)^{-1} e \end{aligned}$$

Dada la forma de la función Φ , podemos ver que si λ crece de modo que $x^k - \lambda^k \nabla f(x^k)$ se acerque a la frontera del poliedro P (es decir, esté muy cerca de la frontera de la restricción, y $b_i - a_i^T(x^k + \lambda d^k)$ sea positivo pero muy cercano a 0), entonces $h(\lambda)$ crecerá hacia infinito, por lo cual será imposible satisfacer las condiciones de Wolfe-Armijo, así que, necesariamente, cualquier punto que satisfaga las condiciones está dentro del poliedro.

Más adelante en el curso veremos que esta función Φ es de gran importancia en el desarrollo de toda una familia de algoritmos alternativos al Simplex, para Programación Lineal. Estos son los Algoritmos de Punto Interior.

2.7.4. Localización espacial

Un problema importante en muchos ámbitos es el de localizar en el plano o el espacio ciertos objetos, los cuales deben cumplir cierta condición con respecto a la distancia a otros que están fijos. Un ejemplo de esto sería la ubicación en alguna zona geográfica de una cantidad de centros de emergencia los cuales deben estar “cerca” de los lugares en los que podrían producirse emergencias.

Vamos a formular este problema de la siguiente forma. Asumiremos que existen m puntos fijos con coordenadas $u^1, \dots, u^m$ conocidas, donde $u^i \in \mathbb{R}^2$ (es decir, $u^i = (u_1^i, u_2^i)$). Tenemos que definir la ubicación de otros K puntos, cuyas coordenadas (que hay que encontrar) denotaremos por $x^1, \dots, x^K$, donde $x^k \in \mathbb{R}^2$ (es decir, $x^k = (x_1^k, x_2^k)$). Existe un costo por unidad de distancia entre el punto fijo i y el punto a localizar k , que denotamos c_{ik} . Queremos que los puntos de coordenadas x^k estén ubicados de modo que, en forma agregada, se minimice un costo total relacionado a la distancia a la que queden de los puntos fijos y los costos por unidad de distancia. Para eso se postula resolver el problema de optimización $\min f(x)$, donde estamos llamando x al vector que forman todos los $x^1, x^2, \dots, x^K$ “pegados” (tenga cuidado con esto: como cada x^k tiene dos coordenadas, x tiene dimensión $2K$), y donde:

$$f(x) = \sum_{k=1}^K \sum_{i=1}^m c_{ik} \|u^i - x^k\|_2^2$$

Esta función es evidentemente convexa así que el problema es susceptible de ser abordado con los métodos que hemos estudiado hasta ahora en el curso. En particular, vamos a considerar el Método del Gradiente para este problema.

Primero, vamos a determinar el gradiente y el hessiano de f en un punto $x \in \mathbb{R}^{2K}$ cualquiera. Se tiene que la función es:

$$f(x) = \sum_{k=1}^K \sum_{i=1}^m c_{ik} \|u^i - x^k\|_2^2$$

Sabemos que por la norma euclideana se puede representar de la forma:

$$f(x) = \sum_{k=1}^K \sum_{i=1}^m c_{ik} (x^k - u^i)^T (x^k - u^i)$$

Derivamos entonces para obtener el gradiente $\nabla f(x)$, siendo un vector de dimensiones $2K \times 1$ donde:

$$\begin{aligned} \frac{\partial f}{\partial x_1^k} &= 2 \sum_{i=1}^m c_{ik} (x_1^k - u_1^i) \\ \frac{\partial f}{\partial x_2^k} &= 2 \sum_{i=1}^m c_{ik} (x_2^k - u_2^i) \end{aligned}$$

$$\nabla f(x^k) = \begin{bmatrix} 0 \\ 0 \\ \dots \\ 1 \\ 1 \\ \dots \\ 0 \\ 0 \end{bmatrix} 2 \sum_{i=1}^m c_{ik} (x^k - u^i)$$

Es decir, la estructura del vector es “por bloques” de dimensión 2, que es cada uno de los vectores del punto k .

Bajo la misma lógica se obtiene el Hessiano $\nabla^2 f(x)$, siendo una Matriz de dimensiones $2K \times 2K$ diagonal, donde:

$$\begin{aligned} \frac{\partial^2 f}{\partial (x_1^k)^2} &= 2 \sum_{i=1}^m c_{ik} \\ \frac{\partial^2 f}{\partial (x_2^k)^2} &= 2 \sum_{i=1}^m c_{ik} \end{aligned}$$

$$\nabla^2 f(x^k) = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & \dots & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & \dots & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} 2 \sum_{k=1}^K \sum_{i=1}^m c_{ik}$$

Ahora queremos estudiar la convergencia del Método del Gradiente aplicado a este problema, para llegar a una solución ϵ -óptima. Como sabemos, el Método de Gradiente tiene convergencia lineal:

$$|f(x^n) - f(x^*)| \leq \alpha |f(x^{n-1}) - f(x^*)|$$

Con n representando la iteración actual.

Siendo $e_n = |f(x^n) - f(x^*)|$, sabemos que eso lleva a:

$$n \geq \frac{1}{|\log \alpha|} \log \frac{e_0}{\epsilon}$$

Al ser la matriz Hessiana definida positiva, se sabe que:

$$\alpha = \left(\frac{k(Q) - 1}{k(Q) + 1} \right)^2$$

Donde $k(Q)$ es el condicionamiento del Hessiano:

$$k(Q) = \frac{\mu_{max}}{\mu_{min}}$$

Con μ_{max} y μ_{min} como el mayor y menor valor propio respectivamente de la matriz Hessiana, que como se dijo en a) depende únicamente de c_{ik} (ver las fórmulas), por lo que el condicionamiento α de la matriz no depende de la ubicación u^i de los locales de emergencia.

Ahora bien, e_0 depende del punto inicial establecido, por lo que eso sí depende de u^i .

$$n \geq \frac{1}{2 \left| \log \left(\frac{k(Q) - 1}{k(Q) + 1} \right) \right|} \log \left(\frac{e_0}{\epsilon} \right)$$

Ahora, una de las etapas del método es la realización del linesearch. Podríamos pensar en hacer algún método como los explicados antes en este capítulo, pero eso no es realmente necesario para este problema. En efecto, tenemos que tomando $\bar{x}$ como la variable genérica en cuestión, se debe determinar de manera óptima el paso:

$$f(\bar{x} + \lambda d)$$

Para esto se define la función objetivo:

$$f(\bar{x} + \lambda d) = \sum_{k=1}^K \sum_{i=1}^m c_{ik} \|(\bar{x} + \lambda d)^k - u^i\|_2^2$$

Entonces para encontrar el paso óptimo derivando en función de λ e igualando a cero.

$$f'(\bar{x} + \lambda d) = 0 = 2 \sum_{k=1}^K \sum_{i=1}^m c_{ik} ((\bar{x} + \lambda d)^k - u^i)^T d^k$$

de donde se despeja λ .

Por último, una observación que queremos hacer ahora es que el problema se puede abordar resolviendo problemas de menor tamaño. Al tener que asignar varias coordenadas a varios puntos según su proximidad óptima a los puntos de emergencia u^i , se podría abordar este problema buscando el óptimo para cada punto x^k de manera individual ya que la función objetivo se separa por el índice k de los puntos. De esta forma, se reduciría el tiempo de ejecución por cada subproblema, pues no se trabajaría con K variables a la vez. De hecho, se podría calcular todo en paralelo.

Esto afectaría, eso sí, las conclusiones anteriores en que el número de iteraciones para la solución ϵ -óptima sería menor. Esto es debido a que al depender de los valores propios de la matriz hessiana, al separar en K subproblemas, estos valores serían más adecuados a la variable (x_1^k, x_2^k) en cuestión del subproblema k

. Esto provoca que la convergencia de cada variable a su óptimo sea más especializada a sí misma, pues no considera los demás valores propios de otras variables irrelevantes, haciendo que el paso de Linesearch pudiera ser más efectivo en cada subproblema.

Recopilando todos los óptimos de cada variable, se esperaría que al resolver el problema de esta forma tarde menos tiempo.

2.7.5. Estimaciones Estadísticas

Se está realizando una investigación sobre los factores que influyen en qué personas de una cierta población adquieran una enfermedad. Lo que se busca es relacionar la propensión al contagio con algunos factores como peso, edad, presión arterial y otros que podrían ser explicativos. Para esto, se dispone de datos de m personas y se ha definido un indicador y donde $y_i = 1$ si la persona contrajo la enfermedad e $y_i = 0$ en caso contrario. Además, se dispone de mediciones de n variables (edad, peso, etc.) para esas m personas, donde u_{ij} es el valor de la variable j para la persona i . Se tiene que $m > n$. Además, en el grupo bajo estudio hay q personas contagiadas y $m - q$ no contagiadas y los datos están ordenados de modo que los contagiados corresponden a las personas $i = 1, \dots, q$ y los no contagiados corresponden a las personas $i = q + 1, \dots, m$. Para este tipo de estudios se puede usar un “modelo logístico”, en el cual se modela la probabilidad de adquirir la enfermedad, p , de la siguiente forma:

$$p = \frac{e^{\alpha^T u}}{1 + e^{\alpha^T u}}$$

donde $\alpha \in \mathbb{R}^n$ corresponde a los parámetros del modelo (a veces hay, además, un término constante pero acá lo estamos obviando para simplificar). Entonces, para la persona i , se estima una probabilidad dada por

$$p_i = \frac{e^{\alpha^T u^i}}{1 + e^{\alpha^T u^i}}, \quad i = 1, \dots, m$$

donde $u^i \in \mathbb{R}^n$ corresponde al vector de las observaciones para la persona i . Es decir, las componentes de u^i son $u_{i1}, u_{i2}, \dots, u_{in}$.

Este problema se aborda buscando los estimadores de “máxima verosimilitud” del modelo, es decir, los valores de los parámetros α que maximizan la probabilidad conjunta de contagio y no contagio en el grupo observado, que es:

$$\prod_{i=1}^q p_i \prod_{i=q+1}^m (1 - p_i)$$

Como esta expresión es compleja, lo que se hace, en forma equivalente, es maximizar la log-verosimilitud (con logaritmo natural) dada por:

$$l(\alpha) = \sum_{i=1}^q \log p_i + \sum_{i=q+1}^m \log(1 - p_i) = \sum_{i=1}^q \alpha^T u^i - \sum_{i=1}^m \log(1 + e^{\alpha^T u^i})$$

(la última fórmula está bien, la primera suma sí es desde 1 a q .) Esta es la función de n variables con la que trabajamos y lo que queremos hacer es resolver el siguiente problema de optimización:

$$\max_{\alpha \in \mathbb{R}^n} l(\alpha)$$

Vamos a discutir cómo los algoritmos de Gradiente, Newton y BFGS funcionarían para un problema de este tipo. Vamos a partir por calcular explícitamente el gradiente y el Hessiano de la función l , ya que son usados por los métodos que queremos estudiar.

Podemos anotar $l(\alpha)$ como:

$$l(\alpha) = \sum_{i=1}^q \sum_{j=1}^n \alpha_j u_{ij} - \sum_{i=1}^m \log(1 + e^{\sum_{j=1}^n \alpha_j u_{ij}})$$

Luego las primeras derivada parciales, para cada j son:

$$\frac{\partial l(\alpha)}{\partial \alpha_j} = \sum_{i=1}^q u_{ij} - \sum_{i=1}^m \frac{u_{ij} e^{\sum_{j=1}^n \alpha_j u_{ij}}}{1 + e^{\sum_{j=1}^n \alpha_j u_{ij}}} = \sum_{i=1}^q u_{ij} - \sum_{i=1}^m \frac{u_{ij} e^{\alpha^T u^i}}{1 + e^{\alpha^T u^i}} \quad j = 1, \dots, n$$

Por otra parte las segundas derivadas son:

$$\frac{\partial^2 l(\alpha)}{\partial \alpha_j \partial \alpha_k} = - \sum_{i=1}^m \frac{u_{ij} u_{ik} e^{\alpha^T u^i} (1 + e^{\alpha^T u^i}) - u_{ij} u_{ik} e^{\alpha^T u^i} e^{\alpha^T u^i}}{(1 + e^{\alpha^T u^i})^2} = - \sum_{i=1}^m \frac{u_{ij} u_{ik} e^{\alpha^T u^i}}{(1 + e^{\alpha^T u^i})^2} \quad j, k = 1, \dots, n$$

Luego para el método del gradiente necesitamos el gradiente que en cada componente esta compuesto por la respectiva derivada parcial.

Para el método de Newton, necesitamos el gradiente y la inversa del Hessiano, el cual es la matriz compuestas por las segundas derivadas parciales.

Para BFGS necesitamos el gradiente, necesitamos la estimación de la inversa del hessiano H^k en cada iteración k , calculada através de los vectores $s_k = x^{k+1} - x^k$, $y_k = \nabla f(x^{k+1}) - \nabla f(x^k)$ y del valor $\rho_k = \frac{1}{y_k^T s_k}$

Además para estos 3 métodos es necesario realizar la etapa de linesearch para conocer la magnitud del paso por las cuales se multiplica la dirección de ascenso para este caso. Este paso lo podemos obtener con backtracking hasta cumplir la condiciones de Wolfe-Armijo. Para esto podemos asumir que podemos recalcular el gradiente un número constante de veces hasta tener un paso lo suficientemente bueno ya que $\frac{\partial l(\alpha + \lambda d^k)}{\partial \lambda} = \nabla l(\alpha + \lambda d^k)^T d^k$.

Ahora, consideremos el esfuerzo computacional necesario para estos tres métodos. Supongamos que queremos llegar a una solución con un error $\epsilon > 0$. Vamos a estimar el esfuerzo computacional total (en flops) para llegar a esa meta. (Aquí podemos asumir conocidas las constantes “escondidas” en algunas expresiones).

Notamos que tanto para el gradiente como para el hessiano debemos calcular $e^{\alpha^T u^i}$ para cada $i = 1, \dots, m$ en las sumatorias, como calcular $e^{\alpha^T u^i}$ toma $\mathcal{O}(n)$ flops para cada i , todos estos valores los podemos precomputar al inicio de cada iteración en $\mathcal{O}(nm)$ flops.

Luego para el gradiente tenemos que $\sum_{i=1}^q u_{ij}$ es un valor fijo que lo podemos precomputar de manera inicial y como $e^{\alpha^T u^i}$ estan precomputados de manera inicial para cada iteración $\sum_{i=1}^q u_{ij} - \sum_{i=1}^m \frac{u_{ij} e^{\alpha^T u^i}}{1 + e^{\alpha^T u^i}}$ se puede calcular de manera eficiente en $\mathcal{O}(m)$ flops, para cada componenete j del gradiente, por lo que calcular el gradiente de manera eficiente se puede realizar en $\mathcal{O}(nm)$ flops. Calcular el gradiente es lo más costoso del método del gradiente, por lo que podemos asumir un costo operacional por iteración de $\mathcal{O}(nm)$ flops para el método del gradiente ya que la etapa de linersearch la podemos asumir que es equivalente a calcular un número constante de veces el gradiente, por lo que el costo por iteración sigue siendo equivalente a $\mathcal{O}(nm)$.

Para Newton, utilizando que $e^{\alpha^T u^i}$ están precalculados (en el gradiente) de manera inicial para cada iteración para cada i , podemos aprovechar esto para calcular cada componente del hessiano en $\mathcal{O}(m)$ flops. Luego como el Hessiano es una matriz de $n \times n$, entonces computar el hessiano toma $\mathcal{O}(n^2 m)$ flops y luego invertir el hessiano toma $\mathcal{O}(n^3)$, por lo que por iteración el método de Newton toma $\mathcal{O}(n^3 + n^2 m)$. (Notamos que linesearch es menos costoso que obtener e invertir el Hessiano, así que no le sumamos costo por iteración).

Para BFGS necesitamos por cada iteración calcular el gradiente y la estimación del Hessiano H^k , una vez obtenido el gradiente, sabemos que se puede calcular de manera eficiente H^k en $\mathcal{O}(n^2)$ flops, por lo que lo más costoso del método es obtener el gradiente y resolver Linersearch, por lo que su costo por iteración es $\mathcal{O}(nm)$

Por último nos debemos fijar en la convergencia de los métodos, donde el método del gradiente tiene una convergencia de $\mathcal{O}(\log(\frac{1}{\epsilon}))$ iteraciones para llegar a una solución con error $\epsilon > 0$. El método de Newton requiere de $\mathcal{O}(\log \log(\frac{1}{\epsilon}))$ iteraciones. BFGS no tiene una estimación sencilla del número de iteraciones, pero debería estar en algún punto intermedio. Por lo tanto, el esfuerzo computacional total para todos los métodos es:

Gradiente: $\mathcal{O}(nm \log(\frac{1}{\epsilon}))$

Newton: $\mathcal{O}((n^3 + n^2 m) \log \log(\frac{1}{\epsilon}))$

BFGS: Algo del tipo $K_1 nm \cdot \log \log(\frac{1}{\epsilon}) \leq \text{Costo Total BFGS} \leq K_2 nm \cdot \log(\frac{1}{\epsilon})$, Con $K_1, K_2 \in \mathbb{R}_+$

2.7.6. Función Banana

El problema a considerar aquí es la optimización de la función Banana de Rosenbrok, en su versión multidimensional. Esta está definida como:

$$f(x) = \sum_{i=1}^{n-1} b(x_{i+1} - x_i^2)^2 + (a - x_i)^2$$

donde $a = 1$, $b = 100$. Vamos a estudiar la aplicación de los métodos de este capítulo a este problema.

1. Lo primero que vamos a hacer es calcular el gradiente y Hessiano de la función y estimar el esfuerzo computacional en “flops” que requiere el cálculo de estos, en función de n .

Tenemos que calcular la derivada parcial de una componente k cualquiera y es mejor escribir f de esta forma:

$$f(x) = \sum_{i=1}^{k-2} b(x_{i+1} - x_i^2)^2 + (a - x_i)^2 + b(x_k - x_{k-1}^2)^2 + (a - x_{k-1})^2 + b(x_{k+1} - x_k^2)^2 + (a - x_k)^2 + \sum_{i=k+1}^{n-1} b(x_{i+1} - x_i^2)^2 + (a - x_i)^2$$

Entonces,

$$\frac{\partial f(x)}{\partial x_k} = \begin{cases} 2b(x_k - x_{k-1}^2) - 4b(x_{k+1} - x_k^2)x_k - 2(a - x_k) & k = 2, \dots, n-1 \\ -4b(x_2 - x_1^2)x_1 - 2(a - x_1) & k = 1 \end{cases}$$

Siguiendo al Hessiano, las derivadas parciales son:

$$\begin{aligned} \frac{\partial^2 f(x)}{\partial x_{k+1} \partial x_k} &= -4b \\ \frac{\partial^2 f(x)}{\partial x_{k-1} \partial x_k} &= -4bx_{k-1} \\ \frac{\partial^2 f(x)}{\partial x_k^2} &= -4bx_{k+1} - 12bx_k^2 + 2 \\ \frac{\partial^2 f(x)}{\partial x_l \partial x_k} &= 0, |l - k| > 2 \end{aligned}$$

Notemos que el Hessiano tiene una estructura muy especial, es lo que se llama una matriz “tridiagonal” ya que todos sus elementos son $= 0$, excepto la diagonal y los inmediatamente contiguos.

Podemos ver que cada derivada parcial requiere calcular una cantidad de flops (contando sólo las multiplicaciones), que es, a lo más, una constante, la más larga de evaluar requiere 6 multiplicaciones, no son n multiplicaciones. Como esto se debe hacer para cada derivada parcial, concluimos que el gradiente se evalúa en $O(n)$ flops.

Por otro lado, para el Hessiano, vemos que las derivadas diagonales y sub y sup diagonales, requieren, cada una, a lo más 5 flops (contamos sólo las multiplicaciones). Tenemos que calcular, a lo más, $3n$ de esas derivadas, no n^2 , como sería en un Hessiano de una función más típica. Concluimos que el esfuerzo para calcular el Hessiano es también $O(n)$.

2. Discutamos ahora el esfuerzo computacional en el que se incurre en la resolución (aproximada) del problema de Linesearch. Esto hay que hacerlo en referencia a alguno de los métodos que se usan para aproximar una solución al problema de linesearch, que es

$$\min_{\lambda > 0} f(x + \lambda d)$$

(no estamos haciendo alusión a una iteración k particular aquí). Vamos a tomar el método de bisección, el cual es simple de desarrollar. Repetimos acá la explicación que está más atrás, recordando que

$$h(\lambda) = f(x + \lambda d)$$

Este método utiliza un oráculo de primer orden (usa el valor de las derivadas de f) y su desempeño es eficaz en la medida que f sea “unimodal”, vale decir, que tiene solo un mínimo local en el intervalo de búsqueda $[t_1, t_3]$, de este modo, se busca el punto t^* , tal que $h'(t^*) \approx 0$. La búsqueda se realiza del siguiente modo (ver Figura 2.5): En primer lugar, se comienza evaluando la derivada en los puntos t_1 y t_3 que definen al intervalo, posteriormente se evalúa la derivada en un punto t_2 intermedio y equidistante de t_1 y t_3 . En caso que $h'(t_2) > 0$, entonces se continúa iterando del mismo modo, pero esta vez buscando entre t_1 y t_2 . Si $h'(t_2) < 0$ entonces se continúa iterando del mismo modo, buscando entre t_2 y t_3 .

Entonces, como el método divide sucesivamente el intervalo de búsqueda a la mitad, eventualmente va a lograr encerrar la solución del problema en un intervalo que puede ser arbitrariamente chico. Acá vamos a asumir que pedimos un error no mayor a δ para la solución encontrada. Esto se logrará después que el método haya reducido el intervalo de búsqueda a un largo $\leq \delta$. Si el largo inicial del intervalo de búsqueda es R , en la siguiente iteración será $R/2$, luego $R/4$, y así. En general, después de k iteraciones de la búsqueda se tendrá que el largo del intervalo es $\frac{R}{2^k}$. Tenemos, entonces, que se requieren a lo más $\log_2 \left(\frac{R}{\delta} \right)$ para lograr la precisión buscada.

Ahora bien, en cada iteración del método se requiere calcular $h'(\lambda) = \nabla f(x + \lambda d)^T d$. Por lo tanto, el costo por iteración viene dado por el costo de evaluar el gradiente, $O(n)$, más el costo de hacer el producto punto, que es también $O(n)$. Luego, el costo por iteración es $O(n)$ y el costo total para llegar a una aproximación con error δ es

$$O \left(n \times \log_2 \left(\frac{R}{\delta} \right) \right)$$

Lo anterior es $O(n)$, si se asume que δ se mantiene fijo siempre. Evidentemente, dependerá de la precisión que se desee el esfuerzo computacional que se gasta en el linesearch. Sin embargo, puede terminar representando una parte importante del tiempo, aun si tomamos un δ no muy exigente, buscando sólo satisfacer una condición de Wolf-Armijo.

3. Siguiendo con el análisis del comportamiento de la función, ahora nos preguntamos si acaso el gradiente $\nabla f(x)$ de la función Banana, para una dimensión específica n , puede cumplir o no la propiedad de Lipschitz.

Recordemos, primero, lo que necesitaríamos: debe existir una constante $L > 0$ tal que

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|, \quad x, y \in \mathbb{R}^n$$

En nuestro caso, f es la función Banana. Tomemos $y = x^*$, el punto donde la función Banana alcanza un mínimo. Tenemos que $\nabla f(x^*) = 0$. Entonces, tendría que cumplirse que:

$$\|\nabla f(x)\| \leq L\|x - x^*\|, \quad x, y \in \mathbb{R}^n$$

Ahora bien, volvamos a las expresiones para las derivadas parciales de f que fueron desarrolladas en la Pregunta 1. Acá las repetimos:

$$\frac{\partial f(x)}{\partial x_k} = \begin{cases} 2b(x_k - x_{k-1}^2) - 4b(x_{k+1} - x_k^2)x_k - 2(a - x_k) & k = 2, \dots, n-1 \\ -4b(x_2 - x_1^2)x_1 - 2(a - x_1) & k = 1 \end{cases}$$

Podemos ver que se puede elegir un vector de forma adecuada de manera tal que $\|\nabla f(x)\|$ crezca de manera muy fuerte. De hecho, seleccionemos x de la siguiente forma, dependiendo de un parámetro t :

$$x_1(t) = t, x_i(t) = 0, i = 2, \dots, n$$

Entonces, tenemos que:

$$\begin{aligned} \frac{\partial f(x)}{\partial x_1} &= 4bt^3 - 2(a - t) \\ \frac{\partial f(x)}{\partial x_2} &= -2bt^2 \end{aligned}$$

y todas las demás son idénticamente cero. Entonces, para este gradiente evaluado en este punto especial $x(t)$, tenemos:

$$\|\nabla f(x(t))\| = \sqrt{(4bt^3 - 2(a-t))^2 + 4b^2t^4}$$

Para t grande, este número es del orden de t^2 . Por otro lado, $\|x(t) - x^*\|$ es del orden de t . Con eso, se concluye que no puede nunca cumplirse lo que buscamos ya que $\|\nabla f(x(t))\|$ puede crecer mucho más rápido que $\|x(t) - x^*\|$ y no podrá estar nunca acotado por $L\|x(t) - x^*\|$, para ningún L . La conclusión final es que ∇f no puede cumplir la condición de Lipschitz que buscamos.

4. Supongamos ahora que supiéramos que los puntos de las iteraciones de los métodos de Gradiente y Newton tienden a concentrarse en, digamos, una región de radio 100 del espacio $\mathbb{R}^n$. Vamos a remitirnos, entonces, a la región $R = \{x \in \mathbb{R}^n : \|x\|_2 \leq 100\}$. Vamos a argumentar que en esa región R , sí se puede decir que $\nabla f(x)$ cumple una condición de Lipschitz, restringida a R . También vamos a estimar esa constante.

Para este argumento vamos a recordar que, para que se cumpla la condición buscada debe pasar que

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|, \quad x, y \in \mathbb{R}^n$$

lo que equivale a que

$$\frac{\|\nabla f(x) - \nabla f(y)\|}{\|x - y\|} \leq L, \quad x, y \in \mathbb{R}^n$$

Ahora bien, el lado izquierdo de esta desigualdad, cuando x está muy cercano a y , “parece” una derivada. Más específicamente, elijamos $x = y + th$, donde h es algún vector con $\|h\| = 1$. Entonces, el lado izquierdo es:

$$\frac{\|\nabla f(y + th) - \nabla f(y)\|}{t}$$

y aquí se ve mucho más claro la forma de derivada. Se puede argumentar, entonces, que, cambiando h por los distintos vectores canónicos, que

$$\|\nabla^2 f(x)\| \leq C(n)L$$

donde $C(n)$ es una constante que depende de n . Para efectos de simplificar, vamos a suponer que $C(n) = 1$ (lo que se puede hacer, cambiando de forma adecuada la norma). Entonces, podemos hacer la siguiente estimación para L :

$$L = \sup_{x \in \mathbb{R}^n} \|\nabla^2 f(x)\|$$

Ya sabemos que ese supremo no existe, por lo que comentamos anteriormente. Ahora bien, sin nos reducen el “universo de interés” a $R = \{x \in \mathbb{R}^n : \|x\|_2 \leq 100\}$, que es un conjunto cerrado y acotado, las componentes de x no podrán crecer arbitrariamente. Más aún, para este problema $\|\nabla f(x)\|$ es una función continua en x y se sabe que toda función continua alcanza sus valores extremos en un conjunto cerrado y acotado (compacto). Entonces, sí podemos estimar:

$$L = \max_{x \in R} \|\nabla^2 f(x)\|_2$$

Esa será la constante de Lipschitz que podríamos querer usar.

El proceso de estimación que acabamos de mostrar podría, de hecho, sobreestimar el valor de la verdadera constante de Lipschitz, pero eso no es un problema demasiado grave, para efectos de muchos usos de la constante (incluyendo lo que veremos en el próximo capítulo), sólo necesitamos una cota superior a esta.

La mejor forma de construir un acotamiento a $\|\nabla^2 f(x)\|_2$ es recurrir a propiedades de normas matriciales. Por ejemplo, si Q es una matriz simétrica cualquiera, una cota que puede usarse es la siguiente:

$$\|Q\|_2 \leq \|Q\|_F$$

donde $\|Q\|_F$ es la norma de Frobenius, dada por

$$\|Q\|_F = \sqrt{\sum_{i=1}^n \sum_{j=1}^n q_{ij}^2} = \text{Tr}(Q^T Q)$$

Otra posibilidad es recurrir a la relación

$$\|Q\|_2 \leq \sqrt{n}\|Q\|_\infty$$

donde

$$\|Q\|_\infty = \max_{i=1,\dots,n} \sum_{j=1}^n |q_{ij}|$$

Una referencia posible para todas estas relaciones es, por ejemplo, Horn, R. A. and Johnson, C. R. *Matrix Analysis*, Cambridge University Press, 1990.

Lo importante de estas relaciones es que, tanto $\|Q\|_F$ como $\|Q\|_\infty$ tienen fórmulas explícitas que dependen de los valores de los elementos de Q . De aquí podemos obtener la cota que buscamos ya que conocemos explícitamente $Q = \nabla^2 f(x)$, donde f es la función Banana, tal como se mostró antes. Tenemos que el Hessiano viene dado por:

$$\begin{aligned} \frac{\partial^2 f(x)}{\partial x_{k+1} \partial x_k} &= -4b \\ \frac{\partial^2 f(x)}{\partial x_{k-1} \partial x_k} &= -4bx_{k-1} \\ \frac{\partial^2 f(x)}{\partial x_k^2} &= -4bx_{k+1} - 12bx_k^2 + 2 \\ \frac{\partial^2 f(x)}{\partial x_l \partial x_k} &= 0, |l - k| > 2 \end{aligned}$$

Notemos que la matriz Hessiana es tridiagonal, es decir, los elementos no nulos son solamente el diagonal de cada fila y sus dos vecinos. Con esto, podemos, entonces, escribir una fórmula explícita para $\|\nabla^2 f(x)\|_F$, por ejemplo (por comodidad notacional, escribiremos el cuadrado de la norma).

$$\|\nabla^2 f(x)\|_F^2 = \sum_{j=1}^n (-4bx_{j+1} - 12bx_j^2 + 2)^2 + \sum_{j=2}^n (4bx_{j-1})^2 + \sum_{j=1}^{n-1} (4b)^2$$

2.7.7. Un algoritmo aproximado con estructura “tipo Newton”

Queremos resolver el problema de minimización restringido $\min_{x \in \mathbb{R}^n} f(x)$, para una función f dos veces continuamente diferenciable. No asumiremos más hipótesis sobre f . Sabemos que podríamos usar el Método de Newton para esto pero, debe recordarse, para que este método funcione correctamente es necesario asegurarse de que en cualquier iteración k , la matriz $\nabla^2 f(x^k)$ sea definida positiva (para que la dirección de avance sea de descenso). Como f no es necesariamente convexa, eso no tendría necesariamente que cumplirse.

Antes de seguir, definiremos un concepto importante: Sea $Q \in \mathbb{R}^{n \times n}$ una matriz simétrica. Denotamos por q_{ij} el elemento fila i , columna j de Q . Decimos que Q es **diagonal dominante** si para cada fila i se tiene que $\sum_{j=1, j \neq i}^n |q_{ij}| < |q_{ii}|$.

Teorema: Si $Q \in \mathbb{R}^{n \times n}$ es una matriz simétrica diagonal dominante, entonces Q es definida positiva.

(**Nota:** La propiedad es sólo suficiente, hay matrices definidas positivas que no son diagonal dominantes.)

Proponemos el siguiente algoritmo, algo como una variante del Método de Newton:

Algoritmo de Newtoncito (para minimizar):**Data:** $x^0 \in \mathbb{R}^n$, f , $k = 0$, $\epsilon > 0$.

```

while  $\|\nabla f(x^k)\|_2 > \epsilon$  do
  if  $\nabla^2 f(x^k)$  no es definida positiva then
    Sea  $B = \nabla^2 f(x^k)$ ;
    for  $i = 1, \dots, n$  do
       $a_i = \sum_{j=1, j \neq i}^n |b_{ij}|$ ;
      if  $a_i > |b_{ii}|$  then
         $b_{ii} = a_i + \eta$ 
      end
    end
  else
     $B = \nabla^2 f(x^k)$ ;
  end
  Sea  $d^k = -B^{-1} \nabla f(x^k)$ ;
  Resolver aproximadamente
      
$$\min_{\lambda \geq 0} f(x^k + \lambda d^k) \quad (\text{etapa de "linesearch"})$$

  Sea  $\lambda^k$  solución óptima de ese problema;
  Sea  $x^{k+1} = x^k + \lambda^k d^k$ ;
   $k = k + 1$ ;
end

```

En este algoritmo, $\eta > 0$ es un número dado, y b_{ij} es el elemento fila i , columna j de la matriz B .

¿Qué hace este procedimiento? El algoritmo apunta a asegurar que el Hessiano sea definido positivo, para aprovechar las propiedades del Método de Newton. En ese sentido, si $\nabla^2 f(x^k)$ resulta no serlo, lo que hace es ajustar los elementos diagonales, de modo que la matriz sea diagonal dominante. Una vez hecho esto, por el teorema mencionado en el enunciado, entonces $\nabla^2 f(x^k)$ es definido positivo. Puede ser razonable usar un algoritmo como este ya que permite trabajar sobre una clase más amplia de funciones, que puede que no cumplan necesariamente las características claves del Método de Newton. Es decir, por medio de la transformación de los elementos diagonales, se garantiza que el Hessiano será definido positivo, y, con ello, el método terminará convergiendo, dado que la dirección será de descenso.

¿Qué podría pasar con las propiedades de convergencia del algoritmo, incluyendo la complejidad (es decir, el orden del número de iteraciones necesarias para llegar a un error dado), comparadas con la del Método de Newton? Muy posiblemente podría no haber convergencia cuadrática, según el grado de perturbación que se induzca en la matriz producto de una potencial transformación a su diagonal. En efecto, si los elementos diagonales resultan ser significativamente diferentes de los originales en sucesivas iteraciones de $\nabla^2 f(x^k)$, entonces es posible que se pierda la convergencia cuadrática, siendo, entonces, peor que el Método de Newton.

Sin embargo, hay ciertas características de f que pueden hacer que la convergencia del algoritmo Newtoncito sea prácticamente igual a la del Método de Newton original (es decir, que hagan, aproximadamente, el mismo número de iteraciones para un mismo caso de problema.) Para que esto ocurra, aún si $\nabla^2 f(x^k)$ no es definida positiva, podría ser que $\nabla^2 f(x^k)$ tenga pocos valores propios negativos, y de serlos, cercanos a cero. Alternativamente, podría tener valores propios positivos, casi en su totalidad. De este modo, $\nabla^2 f(x^k)$ estaría “muy cerca” de ser definida positiva, por lo que las correcciones se ejecutarían pocas veces.

Otra posibilidad es que, aún siendo no definida positiva, esté muy cerca de ser diagonal dominante y la corrección será pequeña, en términos relativos. En cualquiera de las situaciones, la dirección determinada por el algoritmo no será muy diferente a la dirección “pura” de Newton y se podría esperar que las trayectorias de puntos generados sean bastante similares. Esto significará que harán un número similar de iteraciones.

Otra propiedad interesante es que si el algoritmo Newtoncito tiene éxito y eventualmente comienza a converger a un mínimo local de la función f , eventualmente, el algoritmo usará una etapa normal de Newton y no será necesario hacer la corrección de las líneas 3 a 8 del pseudocódigo de arriba. En efecto, dado que

Newtoncito está convergiendo, conforme siguen las iteraciones, el Hessiano se vuelve definido positivo (ya que estamos acercándonos a un mínimo local), ya que se está acercando al mínimo local. También, va mejorando el condicionamiento de la matriz, volviéndose estable con el paso del algoritmo. Es así como, entrada en la etapa final, derechamente no es necesario corregirlo con las líneas 3 a 8 del algoritmo.

Un algoritmo de Descenso por Coordenadas

Queremos resolver el problema $\min_{x \in \mathbb{R}^n} f(x)$, donde f es continuamente diferenciable y convexa, y tenemos que n es muy grande. Como se comentó en clases, en esos casos no se puede calcular explícitamente el vector $\nabla f(x)$ así que se recurre a algunas aproximaciones. Consideremos el algoritmo descrito por el siguiente pseudo-código:

Descenso por Coordenadas (para minimizar):

Data: $x^0 \in \mathbb{R}^n$, f , $k = 0$, $LIMITE$, un escalar > 0 .

while $k < LIMITE$ **do**

 Seleccionar $j_0 \in \{1, \dots, n\}$ en forma aleatoria. Si j_0 es igual al de la selección anterior, repetir.

 Sea $d_{j_0}^k = -\frac{\partial f}{\partial x_{j_0}}(x^k)$;

 Sea $d_j^k = 0, j = 1, \dots, n, j \neq j_0$;

 Resolver aproximadamente

$$\min_{\lambda \geq 0} f(x^k + \lambda d^k) \quad (\text{etapa de "linesearch"})$$

 Sea λ^k solución de ese problema;

 Sea $x^{k+1} = x^k + \lambda^k d^k$;

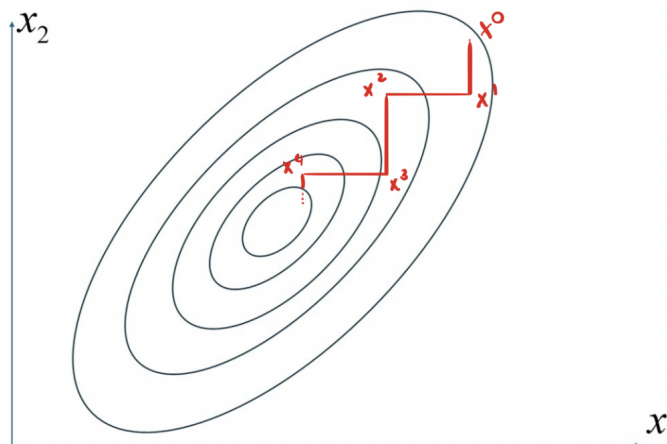
$k = k + 1$;

end

Respecto a la etapa 2 del método, hay una probabilidad de que se repita el índice seleccionado pero, dado que estamos diciendo que n es muy grande, realmente la probabilidad de eso es muy baja y no hay problemas si asumimos que no se repite.

En primer lugar, vamos a mostrar que d^k , calculada por el algoritmo, es una dirección de descenso. En efecto, basta verificar $\nabla f(x^k)^T d^k = -\left(\frac{\partial f(x^k)}{\partial x_{j_0}}\right)^2 < 0$.

Ahora, notemos que lo que este algoritmo hace es tomar aleatoriamente direcciones de descenso paralelas a algún eje, dado que estas son siempre no nulas en sólo una coordenada. Luego, busca el mínimo mediante un Linesearch en esa dirección, y repite el proceso para otra dirección. De ahí es que se le llame “descenso por coordenadas”. La siguiente figura muestra la típica evolución:



Respecto a la posible velocidad de convergencia de este algoritmo, vemos que el método tiene la desventaja de utilizar información parcial del gradiente, que indica la dirección de máximo descenso. Una mala elección de la coordenada j_0 conllevaría a un descenso considerablemente peor que con el método del gradiente, luego

se espera que la cantidad de iteraciones que necesita sea considerablemente mayor. (Claramente esto implica que es mucho peor al compararlo con el Método de Newton en cantidad de iteraciones).

Consideremos ahora la siguiente modificación que viene en el siguiente pseudo-código:

Descenso por Coordenadas Combinado (para minimizar):

Data: $x^0 \in \mathbb{R}^n$, f , $k = 0$, $LIMITE$, un entero > 0 , p , un entero entre 2 y n

while $k < LIMITE$ **do**

$y^1 = x^k$;

for $i = 1, \dots, p-1$ **do**

 Seleccionar $j_0 \in \{1, \dots, n\}$ en forma aleatoria. Si j_0 es igual al de la selección anterior, repetir.

 Sea $h_{j_0}^i = -\frac{\partial f}{\partial x_{j_0}}(y^i)$;

 Sea $h_j^i = 0, j = 1, \dots, n, j \neq j_0$;

 Resolver aproximadamente

$$\min_{\mu \geq 0} f(y^i + \mu h^i) \quad (\text{etapa de "linesearch"})$$

 Sea μ^i solución de ese problema;

 Sea $y^{i+1} = y^i + \mu^i h^i$;

$i = i + 1$;

end

 Sea $d^k = \frac{y^p - x^k}{\|y^p - x^k\|_2}$;

 Resolver aproximadamente

$$\min_{\lambda \geq 0} f(x^k + \lambda d^k) \quad (\text{etapa de "linesearch"})$$

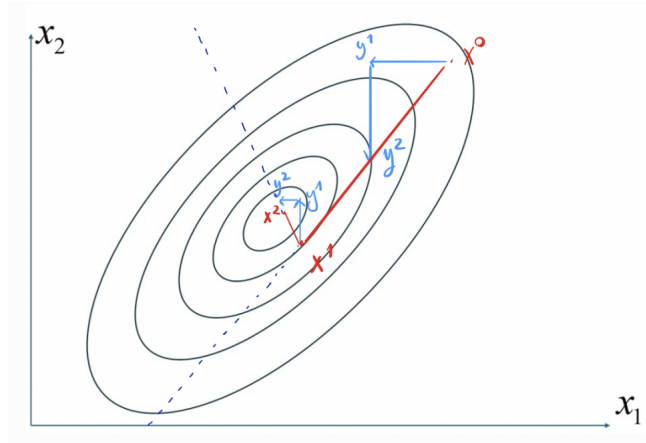
 Sea λ^k solución de ese problema ;

 Sea $x^{k+1} = x^k + \lambda^k d^k$;

$k = k + 1$;

end

El algoritmo extiende el anterior, utilizando más coordenadas para “sondear” el espacio. Esto busca obtener información del entorno de la función para escoger de mejor manera la dirección de descenso, guiándose por donde hay mayor avance “potencial” de la función objetivo, lo cual es, de alguna manera, resumido en la dirección combinada. Gráficamente:



Es esperable que esta modificación, de bajo costo, pueda tener mucho mejor rendimiento que la versión básica del algoritmo.